



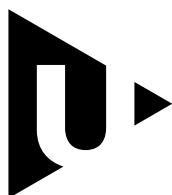
FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

KATEDRA INFORMATIKY
A VÝPOČETNÍ TECHNIKY

Bakalářská práce

Návrh a vytvoření webové aplikace pro pečovatelskou službu

Michal Roučka



**FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI**

**KATEDRA INFORMATIKY
A VÝPOČETNÍ TECHNIKY**

Bakalářská práce

Návrh a vytvoření webové aplikace pro pečovatelskou službu

Michal Roučka

Vedoucí práce

Ing. Jan Rychlík

© Michal Roučka, 2026

Všechna práva vyhrazena. Žádná část tohoto dokumentu nesmí být reprodukována ani rozšiřována jakoukoli formou, elektronicky či mechanicky, fotokopírováním, nahráváním nebo jiným způsobem, nebo uložena v systému pro ukládání a vyhledávání informací bez písemného souhlasu držitelů autorských práv.

Citace v seznamu literatury:

ROUČKA, Michal. *Návrh a vytvoření webové aplikace pro pečovatelskou službu*. Plzeň, 2026. Bakalářská práce. Západočeská univerzita v Plzni, Fakulta aplikovaných věd, Katedra informatiky a výpočetní techniky. Vedoucí práce Ing. Jan Rychlík.

ZÁPADOČESKÁ UNIVERZITA V PLZNI

Fakulta aplikovaných věd

Akademický rok: 2025/2026

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

(projektu, uměleckého díla, uměleckého výkonu)

Jméno a příjmení:	Michal ROUČKA
Osobní číslo:	A23B0140P
Studijní program:	B0613A140037 Softwarové inženýrství
Téma práce:	Návrh a vytvoření webové aplikace pro pečovatelskou službu
Zadávací katedra:	Katedra informatiky a výpočetní techniky

Zásady pro vypracování

1. Seznamte se s organizací práce v prostředí pečovatelské služby.
2. Na základě bodu 1 navrhnete a implementujete software, který bude řešit organizaci práce pečovatelské služby, zejména přidělování práce zaměstnancům a evidenci provedených úkonů.
3. Implementovaný software řádně otestujte.
4. Zhodnoťte dosažené výsledky.

Rozsah bakalářské práce: **doporuč. 30 s. původního textu**
Rozsah grafických prací: **dle potřeby**
Forma zpracování bakalářské práce: **tištěná/elektronická**

Seznam doporučené literatury:

Dodá vedoucí bakalářské práce.

Vedoucí bakalářské práce: **Ing. Jan Rychlík**
Katedra informatiky a výpočetní techniky

Datum zadání bakalářské práce: **30. září 2025**
Termín odevzdání bakalářské práce: **4. května 2026**

L.S.

doc. Ing. Miloš Železný, Ph.D.
děkan

doc. Ing. Přemysl Brada, MSc., Ph.D.
vedoucí katedry

V Plzni dne 30. října 2025

Prohlášení

Prohlašuji, že jsem tuto bakalářskou práci vypracoval samostatně a výhradně s použitím citovaných pramenů, literatury a dalších odborných zdrojů. Tato práce nebyla využita k získání jiného nebo stejného akademického titulu.

Beru na vědomí, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., zákon o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon) v platném znění, a zejména skutečnost, že Západočeská univerzita v Plzni má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona.

V Plzni dne 29. dubna 2026

.....

Michal Roučka

V textu jsou použity názvy produktů, technologií, služeb, aplikací, společností apod., které mohou být ochrannými známkami nebo registrovanými ochrannými známkami příslušných vlastníků.

Během přípravy této práce autor použil nástroj Claude (Anthropic) jako pomocného asistenta při vývoji softwarového řešení – zejména pro návrh dílčích částí kódu, hledání chyb a ověřování implementačních přístupů – a dále při kontrole gramatiky a struktury textu práce. Veškerý kód byl autorem zkontrolován, upraven a začleněn do výsledné aplikace. Obsah a závěry práce jsou výhradně dílem autora, který přebírá plnou odpovědnost za výsledný obsah práce.

Abstrakt

Tato bakalářská práce se zabývá návrhem a implementací webové aplikace CareTracker určené pro správu pečovatelských služeb. Cílem práce je vytvořit komplexní systém pokrývající evidenci klientů a zaměstnanců, zaznamenávání provedených úkonů, správu individuálních plánů péče a generování podkladů pro fakturaci. Systém je navržen jako víceuživatelská platforma s hierarchií přístupových rolí podporující správu více organizací a jejich středisek.

Backend aplikace je implementován pomocí frameworku Spring Boot s využitím relační databáze PostgreSQL. Frontend je realizován jako jednostránková aplikace v prostředí React. Celé prostředí je kontejnerizováno pomocí nástroje Docker.

Výsledkem práce je funkční webová aplikace připravená k nasazení, která pokrývá klíčové procesy pečovatelské organizace a poskytuje přehledné uživatelské rozhraní přizpůsobené různým rolím uživatelů.

Abstract

This bachelor's thesis focuses on the design and implementation of a web application called CareTracker, intended for the management of care services. The goal of the thesis is to develop a comprehensive system covering client and employee records, logging of performed care tasks, management of individual care plans, and generation of billing documents. The system is designed as a multi-user platform with a role-based access hierarchy supporting the management of multiple organizations and their departments.

The application backend is implemented using the Spring Boot framework with a PostgreSQL relational database. The frontend is built as a single-page application using React. The entire environment is containerized using Docker.

The result of this thesis is a fully functional web application ready for deployment, covering the key processes of a care organization and providing a clear user interface tailored to different user roles.

Klíčová slova

pečovatelská služba • webová aplikace • správa péče • Spring Boot • React

Poděkování

Rád bych poděkoval panu Ing. Janu Rychlíkovi za odborné vedení, cenné rady, trpělivost a vstřícnost při konzultacích v průběhu zpracování této bakalářské práce. Dále děkuji své rodině a blízkým za podporu a zázemí, které mi po celou dobu studia poskytovali.

Michal Roučka,
(duben 2026)

Obsah

1	Úvod	4
2	Analýza	6
2.1	Analýza problémové domény	6
2.2	Klíčové procesy pečovatelské organizace	6
2.2.1	Evidence klientů	6
2.2.2	Individuální plány péče	7
2.2.3	Plánování a evidence úkonů	7
2.2.4	Správa zaměstnanců	7
2.2.5	Fakturace	7
2.3	Problémy současného stavu	8
2.4	Analýza existujících řešení	8
2.4.1	Komerční specializované systémy	8
2.4.2	Obecné nástroje	9
2.4.3	Zhodnocení a motivace vlastního řešení	9
2.5	Průzkum mezi pečovatelskými organizacemi	10
2.5.1	Způsob evidence úkonů	10
2.5.2	Administrativní náročnost	10
2.5.3	Správa individuálních plánů péče	11
2.5.4	Zájem o nový systém a akceptovatelná cena	11
2.5.5	Mobilní přístup a bezpečnost dat	12
2.5.6	Závěry průzkumu	12
2.6	Závěr analýzy	12
3	Specifikace	13
3.1	Požadavky na systém	13
3.1.1	Funkční požadavky	13
3.1.2	Nefunkční požadavky	14
3.2	Analýza uživatelů	15
3.2.1	Superadministrátor	15

3.2.2	Administrátor	15
3.2.3	Koordinátor	15
3.2.4	Pečovatel	16
3.2.5	Klient	16
3.2.6	Případy užití	16
3.3	Závěr specifikace	18
4	Technologie	19
4.1	Backend	19
4.1.1	Java a Spring Boot	19
4.1.2	Spring Security	19
4.1.3	Spring Data JPA a Hibernate	19
4.1.4	Flyway	20
4.2	Databáze	20
4.3	Frontend	20
4.3.1	React a JavaScript	20
4.3.2	Vite	20
4.3.3	TailwindCSS a HeroUI	20
4.3.4	React Router	21
4.4	Kontejnerizace	21
4.5	Doplňkové knihovny	21
5	Návrh	22
5.1	Architektura systému	22
5.2	Datový model	22
5.3	Návrh uživatelského rozhraní	25
5.3.1	Přihlašovací stránka	25
5.3.2	Nástěnka	25
5.3.3	Evidence klientů	26
5.3.4	Provedené úkony	26
5.3.5	Individuální plán péče	27
5.3.6	Responzivní design a mobilní rozhraní	30
5.4	Závěr návrhu	31
6	Implementace	32
6.1	Architektura backendu	32
6.2	Zabezpečení a řízení přístupu	34
6.2.1	Filtrování dat podle role	34
6.2.2	Pozvánkový systém	35
6.3	Dynamické filtrování pomocí JPA Specifications	36

6.4	Verzování individuálních plánů péče	36
6.5	Generování PDF	37
6.6	Architektura frontendu	38
6.7	Kontejnerizace	38
7	Testování	40
7.1	Automatizované testy	40
7.1.1	Backendové testy	40
7.1.2	Frontendové testy	40
7.1.3	Výsledky automatizovaných testů	41
7.2	Manuální testování	41
7.2.1	Testovací prostředí	41
7.2.2	Testované scénáře	41
7.3	Závěr testování	42
8	Možnosti dalšího rozvoje	43
9	Závěr	44
A	Dotazník – Průzkum správy pečovatelských služeb	45
B	Uživatelská příručka – Spuštění aplikace	52
B.1	Systémové požadavky	52
B.2	Konfigurace	52
B.3	Spuštění	53
B.4	Přístup k aplikaci	53
B.5	Zastavení a opětovné spuštění	53
	Bibliografie	54
	Seznam obrázků	57
	Seznam tabulek	58
	Seznam výpisů	59

Stárnutí populace představuje jeden z nejvýznamnějších demografických trendů současnosti. Podíl osob starších 65 let na celkové populaci České republiky se dlouhodobě zvyšuje – jak dokládají data Českého statistického úřadu shrnutá v tabulce 1.1 – a v nadcházejících dekáдах se tento trend dále prohloubí. Od roku 2000 do roku 2024 vzrostl podíl seniorů z necelých 14 % na více než 20 % a průměrný věk obyvatele se zvýšil o více než 4 roky. Index stáří, vyjadřující počet osob ve věku 65 a více let na 100 dětí ve věku 0–14 let, vzrostl z 85 na 133 a hranici 100 překročil přibližně v roce 2006 – od té doby je seniorů v populaci více než dětí.

Tabulka 1.1: Vybrané ukazatele věkového složení obyvatelstva České republiky [1]

Rok	Podíl osob 65+ (%)	Průměrný věk (roky)	Index stáří
2000	13,9	38,8	85,5
2002	13,9	39,3	89,2
2004	14,0	39,8	94,0
2006	14,4	40,2	100,2
2008	14,9	40,5	105,1
2010	15,5	40,8	107,8
2012	16,8	41,3	113,3
2014	17,8	41,7	117,4
2016	18,8	42,0	120,7
2018	19,6	42,3	123,2
2020	20,2	42,6	125,5
2022	20,4	42,6	126,1
2024	20,7	43,1	133,4

Spolu s rostoucím počtem osob vyžadujících každodenní pomoc roste i poptávka po profesionálních pečovatelských službách, jejichž úkolem je zajistit klientům důstojný a kvalitní život v jejich přirozeném prostředí. Pečovatelské organizace tak přebírají stále větší část péče, která byla dříve zajišťována převážně rodinou.

Provoz pečovatelské organizace je však administrativně velmi náročný. Koordinátoři musí plánovat a přidělovat úkony desítkám pečovatelů, vést evidenci klientů

včetně jejich zdravotního stavu a individuálních plánů péče, sledovat odpracované úkony a zpracovávat podklady pro fakturaci. Tyto činnosti jsou v mnoha organizacích dosud prováděny manuálně, případně prostřednictvím obecných nástrojů jako jsou tabulkové procesory. Takový přístup je náchylný k chybám, obtížně škálovatelný a neumožňuje efektivní sdílení informací mezi zaměstnanci. Absence specializovaného softwarového řešení tak přímo negativně ovlivňuje kvalitu poskytované péče i ekonomickou efektivitu organizace.

Na trhu sice existují komerční systémy určené pro správu sociálních služeb, avšak jejich nasazení je zpravidla spojeno s vysokými licenčními náklady a obtížnou přizpůsobitelností specifickým potřebám jednotlivých organizací. Menší a středně velké pečovatelské subjekty tak mnohdy nemají přístup k nástrojům, které by odpovídaly jejich provozu a zároveň byly finančně dostupné.

Tato bakalářská práce se zabývá návrhem a implementací webové aplikace *CareTracker*, jejímž cílem je poskytnout pečovatelským organizacím komplexní a přizpůsobitelný nástroj pro správu péče. Systém pokrývá celý životní cyklus pečovatelské služby – od evidence klientů a zaměstnanců, přes zaznamenávání provedených úkonů a správu individuálních plánů péče, až po generování podkladů pro fakturaci. Aplikace je navržena jako víceuživatelský systém s hierarchií přístupových rolí, který umožňuje paralelní správu více organizací a jejich středisek v rámci jedné platformy, přičemž každá organizace má přístup výhradně ke svým vlastním datům.

Důraz je kladen nejen na funkční správnost, ale i na použitelnost systému v reálném provozu. Uživatelské rozhraní je navrženo s ohledem na různorodost uživatelů – od administrátorů a koordinátorů, kteří systém využívají každodenně pro správu a přehled o své organizaci nebo středisku, až po pečovatele, kteří zaznamenávají provedené úkony přímo v terénu prostřednictvím mobilního zařízení. Responzivní design a intuitivní ovládání jsou proto nedílnou součástí návrhu.

Práce je strukturována do několika kapitol. Kapitola 2 obsahuje analýzu požadavků, přehled existujících řešení a vymezení rozsahu projektu. Na jejím základě jsou v kapitole 3 stanoveny funkční a nefunkční požadavky na systém a definovány případy užití. V kapitole 4 jsou představeny zvolené technologie a odůvodněn jejich výběr. Kapitola 5 popisuje architektonický návrh systému, datový model a návrh uživatelského rozhraní. Implementace klíčových částí aplikace je podrobně rozebrána v kapitole 6. Kapitola 7 se věnuje testování a ověření funkčnosti výsledného systému. Kapitola 8 navrhuje možnosti dalšího rozvoje a případná rozšíření aplikace. Závěr práce shrnuje dosažené výsledky a reflektuje přínos realizovaného řešení. Příloha A obsahuje úplné znění dotazníku použitého v analytické fázi; příloha B obsahuje uživatelskou příručku pro instalaci a spuštění aplikace.

2.1 Analýza problémové domény

Pečovatelská služba je jedním ze základních druhů sociálních služeb definovaných zákonem č. 108/2006 Sb., o sociálních službách [2]. Jejím posláním je poskytovat pomoc a podporu osobám, které mají sníženou soběstačnost z důvodu věku, chronického onemocnění nebo zdravotního postižení a jejichž situace vyžaduje pomoc jiné fyzické osoby. Služba je poskytována v přirozeném sociálním prostředí klienta, nejčastěji v jeho domácnosti, a zahrnuje pomoc při zvládnutí běžných úkonů péče o vlastní osobu, pomoc při osobní hygieně, poskytování stravy, pomoc při zajištění chodu domácnosti a zprostředkování kontaktu se společenským prostředím.

Pečovatelské organizace mohou mít různou právní formu – nejčastěji se jedná o příspěvkové organizace zřízené obcí nebo krajem, případně o nestátní neziskové organizace. Větší organizace zpravidla působí na území více obcí a jsou vnitřně členěny na střediska, které pokrývají konkrétní geografické oblasti. Každé středisko má zpravidla vlastního koordinátora, který řídí skupinu pečovatelů a odpovídá za organizaci péče v dané oblasti.

2.2 Klíčové procesy pečovatelské organizace

Chod pečovatelské organizace se opírá o několik vzájemně provázaných procesů, které je nutné řídit souběžně a v koordinaci.

2.2.1 Evidence klientů

Základem každé pečovatelské organizace je evidence klientů. Při přijetí nového klienta jsou zaznamenány jeho osobní údaje, kontaktní informace a stupeň závislosti přiznaný v rámci příspěvku na péči. Klient je zpravidla přiřazen ke konkrétnímu středisku a ke konkrétnímu zaměstnanci, který za jeho péči primárně odpovídá. V průběhu poskytování služby může dojít ke změnám zdravotního stavu klienta,

změně rozsahu poskytované péče nebo ukončení služby, ať již z důvodu zlepšení stavu, přechodu do pobytového zařízení či úmrtí.

2.2.2 Individuální plány péče

Zákon o sociálních službách (§ 88 písm. f) ukládá poskytovateli povinnost zpracovat pro každého klienta individuální plán péče [2]. Tento dokument zachycuje osobní cíle klienta, jeho potřeby a konkrétní způsob, jakým bude péče poskytována. Individuální plán je pravidelně aktualizován a slouží jako závazný podklad pro práci pečovatele. Vedení a aktualizace těchto plánů je administrativně náročná, zejména při větším počtu klientů.

2.2.3 Plánování a evidence úkonů

Stěžejní každodenní agendou je plánování a zaznamenávání provedených úkonů. Každý úkon odpovídá konkrétní službě ze schváleného ceníku – například pomoci při hygieně, přípravě stravy nebo doprovodu k lékaři. Úkony se liší způsobem účtování: některé jsou účtovány hodinově, jiné za jednotlivé úkony, případně za kilogramy nebo kilometry. Pečovatel musí každý provedený úkon zaznamenat včetně času, rozsahu a případně dalších účastníků. Tato data jsou následně podkladem pro vyúčtování vůči klientovi i pro interní kontrolu.

2.2.4 Správa zaměstnanců

Organizace musí vést evidenci svých zaměstnanců, jejich přiřazení ke střediskům a jejich rolí v rámci systému. Pečovatelé jsou přiřazováni k jednotlivým střediskům a mohou být přiřazeni ke klientům jako klíčoví pracovníci. Koordinátoři dohlíží na jejich práci a starají se o zaměstnance svého střediska, zatímco správci mají přístup k datům celé organizace. Správa přístupových práv a přiřazení zaměstnanců je proto nedílnou součástí provozu.

2.2.5 Fakturace

Na základě zaznamenaných úkonů organizace pravidelně – zpravidla měsíčně – vystavuje klientům vyúčtování za poskytnuté služby. Výpočet ceny vychází z ceníku a rozsahu provedených úkonů. Příprava podkladů pro fakturaci manuálním způsobem je časově náročná a vyžaduje přesnost, neboť chyby v účtování mohou mít přímý dopad jak na klienta, tak na hospodaření organizace.

2.3 Problémy současného stavu

Přestože pečovatelské služby plní nezastupitelnou společenskou roli, jejich administrativní procesy jsou v mnoha případech řešeny zdlouhavým a neefektivním způsobem. Každý pečovatel vede pro své klienty tištěný měsíční výkaz v podobě tabulky, do níž zaznamenává čas strávený jednotlivými úkony a počet setkání s klientem. Na konci měsíce je nutné všechny záznamy manuálně sečíst a výkazy předat koordinátorovi, který výsledné součty ručně přepisuje do počítačového systému. Z něj pak zpětně doplňuje do výkazu celkovou částku k úhradě. Celý proces je vysoce repetitivní, náchylný k chybám a může v součtu zabrat desítky hodin práce měsíčně.

Významnou součástí dokumentace jsou individuální plány péče, které představují povinnou evidenci pro každého klienta. Individuální plán je živý dokument, který se průběžně mění a doplňuje tak, aby všichni pečovatelé měli aktuální přehled o průběhu péče a případných změnách. Vedení těchto plánů v papírové podobě komplikuje sdílení informací mezi pečovateli a zvyšuje riziko, že někteří z nich budou pracovat s neaktuálními údaji.

Digitalizace těchto procesů prostřednictvím specializovaného softwarového řešení má proto potenciál výrazně snížit administrativní zátěž, eliminovat chybovost způsobenou ručním přepisováním a zajistit, že všichni pečovatelé mají vždy přístup k aktuálním informacím o svých klientech.

2.4 Analýza existujících řešení

Před zahájením vývoje vlastního systému byla provedena analýza existujících softwarových nástrojů, které jsou v prostředí pečovatelských služeb v České republice využívány. Cílem bylo identifikovat jejich silné stránky, omezení a případné mezery, které by vlastní řešení mohlo zaplnit. Poznatky z analýzy byly doplněny a ověřeny výsledky dotazníkového průzkumu popsaneho v sekci 2.5.

2.4.1 Komerční specializované systémy

Na českém trhu existuje několik komerčních systémů určených přímo pro správu sociálních služeb. Mezi nejrozšířenější patří například *Cygnus* [3] nebo *Pečovatelská služba* [4], které nabízejí komplexní funkcionalitu zahrnující evidenci klientů, zaznamenávání úkonů i fakturaci. Tyto systémy jsou zpravidla provozovány formou SaaS (Software as a Service, česky software jako služba) a jsou průběžně aktualizovány v souladu s legislativními požadavky.

Jejich hlavní nevýhodou jsou vysoké licenční náklady, které mohou být pro menší organizace obtížně dostupné. Systémy bývají navrženy pro velké poskyтова-

tele s množstvím funkcí, které menší organizace nepotřebují, a jejich uživatelské rozhraní může být zbytečně komplexní. Možnosti přizpůsobení specifickým potřebám konkrétní organizace jsou zpravidla omezené a závislé na ochotě dodavatele provést úpravy na zakázku.

Samostatnou kapitolou je správa individuálních plánů péče, která bývá v mnoha systémech opomíjena nebo řešena jen okrajově. Poskytovatelé sociálních služeb jsou ze zákona povinni individuální plány vést a pravidelně aktualizovat, přesto řada dostupných systémů tuto agendu buď nepodporuje vůbec, nebo nabízí pouze základní evidenci bez možnosti strukturovaného vedení denních záznamů a historie změn. Organizace jsou pak nuceny využívat samostatné specializované nástroje, jako je například aplikace *Joplan* [5], která je přímo zaměřena na tvorbu a správu individuálních plánů. Provoz dvou oddělených systémů však přináší nejen dodatečné náklady, ale i komplikace při přenosu dat a riziko nekonzistence informací mezi oběma platformami.

2.4.2 Obecné nástroje

Část organizací, zejména menších, využívá pro správu pečovatelských služeb obecné kancelářské nástroje. V některých organizacích není digitální řešení využíváno vůbec – evidence klientů, úkonů i individuálních plánů péče probíhá výhradně v papírové podobě, přičemž ruční zpracování výkazů může zabrat i desítky hodin měsíčně.

Tabulkové procesory a papírová evidence neumožňují souběžnou práci více uživatelů bez rizika konfliktů, nenabízejí žádný systém přístupových práv a nejsou schopny automaticky provádět výpočty ani generovat strukturované výstupy pro fakturaci. Chybí také možnost propojení dat napříč agendami – například vazba mezi klientem, jeho úkony a individuálním plánem péče. Riziko neúmyslného smazání nebo přepsání dat je vysoké a sdílení informací mezi pracovníky v reálném čase je znemožněno.

2.4.3 Zhodnocení a motivace vlastního řešení

Z provedené analýzy vyplývá, že existující komerční řešení jsou sice funkčně bohatá, avšak pro menší organizace finančně a provozně náročná. Obecné nástroje a papírová evidence naopak představují vysokou administrativní zátěž a nezaručují datovou integritu ani bezpečnost. Na trhu tak existuje prostor pro otevřené, přizpůsobitelné řešení, které pokrývá klíčové potřeby pečovatelské organizace bez zbytečné složitosti a s důrazem na jednoduchost nasazení.

Vlastní vývoj systému CareTracker je motivován právě tímto zjištěním. Cílem není replikovat veškerou funkcionalitu komerčních systémů, ale poskytnout funkčně ucelenou platformu, která pokrývá nejdůležitější procesy – evidenci klientů a zaměstnanců, zaznamenávání úkonů, správu individuálních plánů a přípravu pod-

kladů pro fakturaci – a která je snadno nasaditelná díky kontejnerizaci pomocí Docker.

2.5 Průzkum mezi pečovatelskými organizacemi

Za účelem ověření závěrů provedené analýzy a získání konkrétních poznatků z praxe byl realizován dotazníkový průzkum mezi poskytovateli pečovatelských služeb. Dotazník byl rozeslán elektronickou formou a zaměřoval se na způsob evidence úkonů, administrativní zátěž, spokojenost se současným řešením a zájem o nový systém. Průzkumu se zúčastnilo celkem 11 organizací různé velikosti. Dotazník je k dispozici v příloze A.

2.5.1 Způsob evidence úkonů

Průzkum ukázal, že specializovaný software využívá většina dotázaných organizací, avšak řada z nich jej kombinuje s papírovou evidencí. Čistě papírovou evidenci bez jakéhokoli softwarového nástroje využívá jedna organizace. Přehled způsobů evidence shrnuje tabulka 2.1.

Tabulka 2.1: Způsob aktuální evidence úkonů

Způsob evidence	Počet organizací
Specializovaný software	5
Specializovaný software + papírová evidence	5
Papírová evidence	1

Mezi zmíněnými softwarovými nástroji se nejčastěji objevoval systém od autora pana Zajíce (4 organizace), dále systémy Cygnus (2), eQuip [6] (2) a Pečovatelka Deluxe [7] (1). Kombinace specializovaného softwaru s papírovou evidencí svědčí o tom, že stávající nástroje nepokrývají veškeré potřeby organizací, a část agendy tak zůstává vedena ručně.

2.5.2 Administrativní náročnost

Respondenti měli možnost označit více oblastí, které považují za administrativně nejnáročnější. Přehled shrnuje tabulka 2.2. Nejčastěji bylo zmiňováno plánování směn (6 organizací) a příprava podkladů pro fakturaci (5 organizací), následovaná správou individuálních plánů péče (4 organizace) a sdílením dat v terénu (2 organizace). Individuální plány péče jako zdroj administrativní zátěže potvrzují závěr z analýzy existujících řešení – tato agenda je v dostupných nástrojích řešena nedostatečně.

Tabulka 2.2: Administrativně nejnáročnější oblasti

Oblast	Počet organizací
Plánování směn	6
Příprava podkladů pro fakturaci	5
Správa individuálních plánů péče	4
Sdílení dat v terénu	2

Počet hodin měsíčně strávených ručním zpracováním výkazů se mezi organizacemi výrazně liší. U organizací využívajících výhradně papírovou evidenci dosahuje až 40 hodin měsíčně, u organizací se specializovaným softwarem se pohybuje v rozmezí 0–16 hodin. Průměrná hodnota u respondentů, kteří tento údaj uvedli, činí přibližně 12 hodin měsíčně.

2.5.3 Správa individuálních plánů péče

Způsob vedení individuálních plánů péče je znázorněn v tabulce 2.3. Téměř třetina organizací stále vede individuální plány výhradně v papírové podobě, přičemž část z těch, kteří je vedou digitálně, využívá separátní software oddělený od systému pro evidenci úkonů.

Tabulka 2.3: Způsob vedení individuálních plánů péče

Způsob vedení	Počet organizací
Digitálně – součást softwaru pro úkony	4
Digitálně – separátní software	2
Papírově	3
Přechodné období (papír i digitálně)	1
Neuvedeno	1

2.5.4 Zájem o nový systém a akceptovatelná cena

Na otázku, zda by organizace uvítaly nasazení nového systému, odpovědělo kladně 6 respondentů, 2 zájem nevyjádřili a zbývající odpovědi byly neutrální nebo podmíněné vyzkoušením. Jako největší přínos oproti současnému řešení respondenti nejčastěji uváděli jednoduchost ovládání, automatické generování podkladů pro fakturaci a lepší přizpůsobení potřebám pečovatelské služby.

Nejčastěji akceptovatelná měsíční cena se pohybuje v rozmezí 1 500–3 000 Kč (6 organizací). Jako hlavní rizika při přechodu na nový systém respondenti označili migraci dat a školení zaměstnanců.

2.5.5 Mobilní přístup a bezpečnost dat

Sedm z jedenácti organizací potvrdilo, že jejich pečovatelé mají při výkonu práce k dispozici chytrý telefon s přístupem k internetu. Důležitost možnosti zadávat úkony z mobilního zařízení přímo u klienta hodnotili respondenti průměrnou známkou 3,9 z 5, přičemž pět organizací ji ohodnotilo nejvyšší hodnotou. Z hlediska zabezpečení dat preferuje 6 organizací cloudové řešení, 2 organizace preferují provoz na vlastním serveru a 3 organizace nemají preference.

2.5.6 Závěry průzkumu

Průzkum potvrdil závěry analýzy a přinesl konkrétní data z praxe. Většina organizací využívá specializovaný software, avšak často jej kombinuje s papírovou evidencí, což svědčí o mezerách v pokrytí funkcionality. Správa individuálních plánů péče zůstává problematickou oblastí – téměř třetina organizací ji stále řeší papírově. Ruční zpracování výkazů představuje značnou časovou zátěž a zájem o komplexnější řešení mezi organizacemi existuje. Průzkum rovněž potvrdil důležitost mobilního přístupu a preference cloudového hostingu, které se promítly do nefunkčních požadavků systému CareTracker.

2.6 Závěr analýzy

Provedená analýza ukázala, že správa pečovatelských služeb je administrativně náročná oblast, která v praxi trpí nedostatečnou softwarovou podporou. Komerční systémy jsou funkčně bohaté, avšak finančně a provozně náročné, a často ani nepokrývají celou potřebnou agendu – zejména správu individuálních plánů péče. Obecné nástroje naopak nenabízejí dostatečnou integritu dat ani bezpečnost.

Na základě analýzy byl vymezen rozsah systému CareTracker. Implementovány budou všechny klíčové funkční oblasti: správa organizací, středisek a uživatelů s hierarchií rolí, evidence klientů a zaměstnanců, katalog úkonů, evidence provedených úkonů, správa individuálních plánů péče včetně denních záznamů a generování podkladů pro fakturaci ve formátu PDF. Mimo rozsah této práce zůstávají funkce jako integrace s externími účetními systémy, automatické rozesílání faktur nebo mobilní nativní aplikace – tyto oblasti představují možná rozšíření do budoucna.

Výstupem analýzy je sada funkčních a nefunkčních požadavků podrobně popsaných v kapitole 3, které tvoří základ pro návrh a implementaci systému popsané v následujících kapitolách.

3.1 Požadavky na systém

Na základě analýzy problémové domény a existujících řešení byly stanoveny požadavky na vyvíjený systém. Požadavky jsou rozděleny na funkční, popisující, co systém musí umět, a nefunkční, popisující kvalitativní vlastnosti systému.

3.1.1 Funkční požadavky

3.1.1.1 FR-01: Správa organizací a středisek

Systém musí podporovat provoz více organizací v rámci jedné platformy, přičemž každá organizace má přístup výhradně ke svým datům. V rámci organizace musí být možné spravovat střediska, která představují jednotlivé pobočky nebo geograficky vymezené oblasti působení.

3.1.1.2 FR-02: Správa uživatelů a přístupových rolí

Systém musí rozlišovat několik úrovní přístupu odpovídajících reálné hierarchii pečovatelské organizace. Superadministrátor spravuje celou platformu napříč organizacemi. Administrátor spravuje svou organizaci a její zaměstnance. Koordinátor řídí péči v rámci svého střediska a spravuje klienty. Pečovatel zaznamenává provedené úkony a má přístup k datům svých klientů. Klient má přístup pouze ke svým vlastním datům v režimu čtení. Každý uživatel musí mít možnost přihlásit se do systému a provádět pouze operace odpovídající své roli.

3.1.1.3 FR-03: Evidence klientů

Systém musí umožňovat vedení evidence klientů včetně jejich osobních údajů, přiřazení ke středisku a pečovateli, stupně závislosti a stavu smlouvy. Musí být možné klienta deaktivovat při ukončení poskytování služby a zaznamenat důvod a datum ukončení.

3.1.1.4 FR-04: Evidence zaměstnanců

Systém musí umožňovat správu zaměstnanců včetně jejich přiřazení k organizaci a středisku a nastavení přístupové role. Musí být možné zaměstnance deaktivovat bez ztráty historických dat.

3.1.1.5 FR-05: Katalog úkonů

Organizace musí mít možnost spravovat vlastní katalog úkonů určených k fakturaci. Každý úkon má název, jednotkovou cenu a typ účtování – hodinový, kusový, kilogramový nebo kilometrový. Katalog slouží jako základ pro evidenci provedených úkonů a výpočet ceny.

3.1.1.6 FR-06: Evidence provedených úkonů

Systém musí umožňovat zaznamenávání provedených úkonů s vazbou na klienta, pečovatele, datum a rozsah. Jeden úkon může být proveden více pečovateli současně. Koordinátor musí mít přehled o všech úkonech ve svém středisku. Systém musí umožňovat filtrování úkonů podle organizace, střediska, pečovatele, klienta a období.

3.1.1.7 FR-07: Individuální plány péče

Systém musí umožňovat vedení individuálních plánů péče pro každého klienta. Plán musí obsahovat denní záznamy popisující průběh péče. Musí být podporováno verzování, aby byla zachována historie změn plánu v čase.

3.1.1.8 FR-08: Fakturace

Systém musí umožňovat generování podkladů pro fakturaci na základě zaznamenaných úkonů za zvolené období. Výstupem musí být přehled úkonů s vypočtenými cenami ve formátu vhodném pro tisk nebo export (PDF).

3.1.2 Nefunkční požadavky

3.1.2.1 NFR-01: Bezpečnost

Přístup do systému musí být chráněn autentizací. Každý uživatel smí přistupovat pouze k datům, ke kterým má oprávnění odpovídající jeho roli a organizaci. Aplikace musí být odolná vůči běžným webovým hrozbám, zejména SQL injection, XSS a CSRF útokům.

3.1.2.2 NFR-02: Použitelnost

Uživatelské rozhraní musí být responzivní a použitelné na mobilních zařízeních, neboť pečovatelé zadávají úkony zpravidla v terénu. Ovládání musí být intuitivní a nevyžadovat rozsáhlé zaškolení.

3.1.2.3 NFR-03: Nasaditelnost

Celá aplikace musí být kontejnerizována pomocí Docker a Docker Compose tak, aby bylo možné ji spustit na libovolném serveru bez složité konfigurace závislostí. Databázové schéma musí být verzováno a migrace automatizovány.

3.1.2.4 NFR-04: Rozšiřitelnost

Architektura systému musí být navržena tak, aby bylo možné v budoucnu přidávat nové funkce bez zásadních změn stávající struktury. Backend musí být oddělen od frontendu prostřednictvím REST API.

3.2 Analýza uživatelů

Systém je navržen pro heterogenní skupinu uživatelů s různými potřebami a úrovněmi technické zdatnosti. Pro správné vymezení funkcionality a přístupových práv bylo nutné identifikovat jednotlivé role a jejich typické způsoby použití systému.

3.2.1 Superadministrátor

Superadministrátor je globální správce celé platformy. Jeho úkolem je zakládat nové organizace a spravovat jejich administrátory. Typicky nepracuje s daty konkrétních klientů ani úkonů – jeho agenda je čistě správní a konfigurační. Tuto roli zastávají zpravidla provozovatelé platformy.

3.2.2 Administrátor

Administrátor spravuje jednu konkrétní organizaci. Zakládá střediska, spravuje zaměstnance a nastavuje katalog úkonů. Má přehled o všech datech v rámci své organizace a může zasahovat do dat libovolného střediska. Tuto roli zastává zpravidla vedení organizace nebo její IT správce.

3.2.3 Koordinátor

Koordinátor je klíčovou operativní rolí. Spravuje klienty, úkony a zaměstnance v rámci svého střediska a dohlíží na průběh péče. Pravidelně pracuje se systémem v kancelářském prostředí, a proto jsou pro něj důležité přehledové funkce, filtrování

a možnost exportu dat pro fakturaci. Koordinátor musí mít rychlý přístup k aktuálnímu stavu úkonů za dané období a k individuálním plánům péče svých klientů.

3.2.4 Pečovatel

Pečovatel zaznamenává provedené úkony bezprostředně po jejich vykonání, často prostřednictvím mobilního zařízení v domácnosti klienta. Pracuje s omezenou množinou dat – vidí pouze klienty a úkony ve svém středisku. Pro tuto roli je klíčová jednoduchost a rychlost ovládání. Rozhraní musí být plně funkční na malé obrazovce a nevyžadovat složitou navigaci.

3.2.5 Klient

Klient má přístup do systému v režimu čtení. Může nahlížet na záznamy o péči, která mu byla poskytnuta, a zobrazit si QR kód pro platbu za služby. Tato role existuje za účelem transparentnosti vůči příjemcům péče a jejich rodinám. Klient nemůže žádná data upravovat ani vytvářet.

3.2.6 Případy užití

Na základě identifikovaných rolí byly definovány klíčové případy užití systému. Každý případ užití popisuje konkrétní interakci uživatele se systémem a slouží jako základ pro specifikaci funkčních požadavků.

Správa organizací je dostupná pouze superadministrátorovi, který zakládá organizace a přiřazuje jim administrátory. Administrátor spravuje střediska v rámci své organizace. Správa zaměstnanců a nastavení jejich rolí přísluší administrátorovi a koordinátorovi v rozsahu odpovídajícím jejich středisku. Evidence klientů je v kompetenci koordinátora, zatímco zaznamenávání provedených úkonů a správu individuálních plánů provádí převážně pečovatel. Generování podkladů pro fakturaci je dostupné všem zaměstnancům.

Diagram na obrázku 3.1 zachycuje přehled všech případů užití a jejich vazby na jednotlivé role. Vztah «*include*» je využit u use cases *Pozvání koordinátora*, *Pozvání pečovatele*, *Pozvání klienta* a *Reset hesla*, které všechny povinně zahrnují *Odeslání e-mailové pozvánky* – tato akce je nedílnou součástí každého z nich a probíhá automaticky při jejich spuštění.



Obrázek 3.1: Diagram případů užití

3.3 Závěr specifikace

Na základě analýzy uživatelů a jejich potřeb byly definovány funkční a nefunkční požadavky na systém a identifikovány klíčové případy užití. Systém bude sloužit pěti typům uživatelů s odlišnými přístupovými právy a pokryje všechny hlavní procesy pečovatelské organizace – evidenci klientů a zaměstnanců, zaznamenávání provedených úkonů, správu individuálních plánů péče a generování podkladů pro fakturaci.

Všechny definované funkční požadavky budou v rámci této práce implementovány. Architektura systému bude navržena s ohledem na nefunkční požadavky, zejména bezpečnost, responzivitu a snadnou nasaditelnost. Návrh systému vycházející z této specifikace je popsán v následující kapitole.

Výběr technologií byl řízen požadavky definovanými ve specifikaci – zejména potřebou spolehlivého backendového frameworku s dobrou podporou bezpečnosti, moderního frontendového řešení s responzivním uživatelským rozhraním a snadné nasaditelnosti prostřednictvím kontejnerizace. Tato kapitola představuje zvolené technologie a odůvodňuje jejich výběr.

4.1 Backend

4.1.1 Java a Spring Boot

Backend aplikace je implementován v jazyce Java 21 [8] s využitím frameworku Spring Boot 3 [9]. Spring Boot byl zvolen pro svoji rozsáhlou dokumentaci a přirozenou integraci bezpečnostní vrstvy, přístupu k datům a REST API v rámci jednotného ekosystému. Díky automatické konfiguraci nevyžaduje rozsáhlé ruční nastavení a umožňuje rychlé vytvoření produkčně připravené aplikace. Alternativou byl micro-framework Flask v jazyce Python [10], avšak ten vyžaduje ruční integraci jednotlivých komponent – bezpečnostní vrstvy, přístupu k datům i správy schématu. Spring Boot tyto komponenty integruje nativně, což urychluje vývoj a snižuje riziko chyb při jejich propojování.

4.1.2 Spring Security

Autentizace a autorizace jsou řešeny pomocí Spring Security [11]. Pro systém CareTracker bylo zvoleno přihlašování prostřednictvím session uložené v cookie s platností 7 dní, které je přirozené pro webové aplikace. Spring Security dále zajišťuje ochranu před běžnými webovými útoky.

4.1.3 Spring Data JPA a Hibernate

Přístup k databázi je realizován prostřednictvím Spring Data JPA [12] s Hibernate [13] jako implementační vrstvou zajišťující převod mezi objekty v Javě a zá-

znamy v databázi. Spring Data JPA výrazně snižuje množství potřebného kódu pro standardní databázové operace. Pro složitější dynamické dotazy – například filtrování úkonů podle více kritérií – je využito rozhraní JPA Specifications.

4.1.4 Flyway

Správa databázového schématu je verzována pomocí nástroje Flyway [14]. Změny schématu jsou definovány jako sekvence SQL skriptů, které jsou při startu aplikace automaticky aplikovány. Tím je zaručena konzistence databáze s verzí aplikace a umožněno reprodukovatelné nasazení. Oproti alternativnímu nástroji Liquibase byl Flyway zvolen pro svoji jednoduchost.

4.2 Databáze

Jako relační databázový systém byl zvolen PostgreSQL 15 [15]. Oproti alternativám jako MySQL nebo MariaDB nabízí PostgreSQL lepší podporu komplexních dotazů a referenční integrity, což je pro datově provázanou doménu pečovatelské organizace důležité.

4.3 Frontend

4.3.1 React a JavaScript

Frontend je implementován jako jednostránková aplikace s využitím knihovny React 19 [16] v jazyce JavaScript [17]. Část vstupních souborů a kontextů využívá TypeScript [18], avšak drtivá většina kódu je psána v JavaScriptu s JSX syntaxí. React byl zvolen pro svoji rozšířenost, velký ekosystém a širokou komunitu. Alternativami byly Vue.js nebo Angular, avšak React nabízí nejširší výběr podpůrných knihoven.

4.3.2 Vite

Jako nástroj pro sestavení frontendu a vývojový server byl zvolen Vite [19]. Oproti tradičním nástrojům jako Webpack dosahuje Vite výrazně rychlejšího startu vývojového serveru a okamžité aktualizace změn v prohlížeči bez nutnosti obnovení stránky.

4.3.3 TailwindCSS a HeroUI

Stylování je realizováno pomocí TailwindCSS v4 [20], CSS frameworku postaveného na sadě předdefinovaných tříd, které lze kombinovat přímo v HTML. Tento přístup zaručuje konzistentní vizuální styl napříč celou aplikací a dobrou podporu

responzivního designu. Na TailwindCSS je postavena komponentová knihovna HeroUI [21], která poskytuje sadu předpřipravených React komponent – formulářové prvky, tabulky, modální okna a navigační prvky – a umožňuje tak rychle sestavit funkční rozhraní bez implementace základních UI komponent od základů.

4.3.4 React Router

Navigace a ochrana tras jsou zajištěny pomocí React Router v7 [22]. Přístup k jednotlivým trasám aplikace je podmíněn rolí přihlášeného uživatele.

4.4 Kontejnerizace

Celé prostředí aplikace je kontejnerizováno pomocí Dockeru [23] a orchestrováno nástrojem Docker Compose [24]. Každá vrstva – backend, frontend a databáze – běží ve vlastním izolovaném kontejneru. Docker Compose zajišťuje správné pořadí spouštění služeb, propojení vnitřní sítě a správu perzistentních datových svazků. Díky kontejnerizaci je možné celou aplikaci nasadit na libovolný server jediným příkazem bez instalace závislostí.

4.5 Doplnkové knihovny

Generování PDF dokumentů pro fakturační podklady je realizováno pomocí knihovny Flying Saucer [25], která převádí HTML stylizované pomocí CSS do formátu PDF. Tento přístup byl zvolen pro snadnou úpravu šablon dokumentů. Odesílání e-mailových pozvánek nových uživatelů je implementováno pomocí Spring Mail [26] se šablonami Thymeleaf [27].

Tato kapitola popisuje návrh systému CareTracker – jeho architekturu, datový model a uživatelské rozhraní. Návrh vychází z požadavků definovaných ve specifikaci a byl průběžně upřesňován v průběhu implementace.

5.1 Architektura systému

Systém je postaven na třívrstvé architektuře rozdělené na prezentační vrstvu, aplikační vrstvu a datovou vrstvu.

Prezentační vrstva je tvořena jednostránkovou webovou aplikací v Reactu, která běží v prohlížeči uživatele. Uživatel pracuje výhradně s touto vrstvou – zadává data do formulářů, prohlíží přehledy a spouští akce. Veškerá komunikace s backendem probíhá prostřednictvím REST API ve formátu JSON.

Aplikační vrstva je implementována jako backendová aplikace ve Spring Bootu. Zpracovává požadavky od frontendu, ověřuje oprávnění uživatele, provádí výpočty a ukládá nebo načítá data z databáze. Backend rovněž zajišťuje generování PDF dokumentů a odesílání e-mailů.

Datová vrstva je tvořena relační databází PostgreSQL, která uchovává veškerá data systému. Přístup k databázi je realizován výhradně přes aplikační vrstvu – frontend nemá přímý přístup k datům.

Všechny tři vrstvy jsou provozovány jako samostatné Docker kontejnery orchestrované nástrojem Docker Compose, díky čemuž je celé prostředí snadno přenositelné a spustitelné na libovolném serveru.

5.2 Datový model

Datový model systému je znázorněn na obrázku 5.1. Centrální entitou je *Organization*, která reprezentuje jednoho poskytovatele pečovatelské služby. V rámci organizace jsou definována střediska (*Department*), zaměstnanci (*Employee*) a klienti (*Client*).

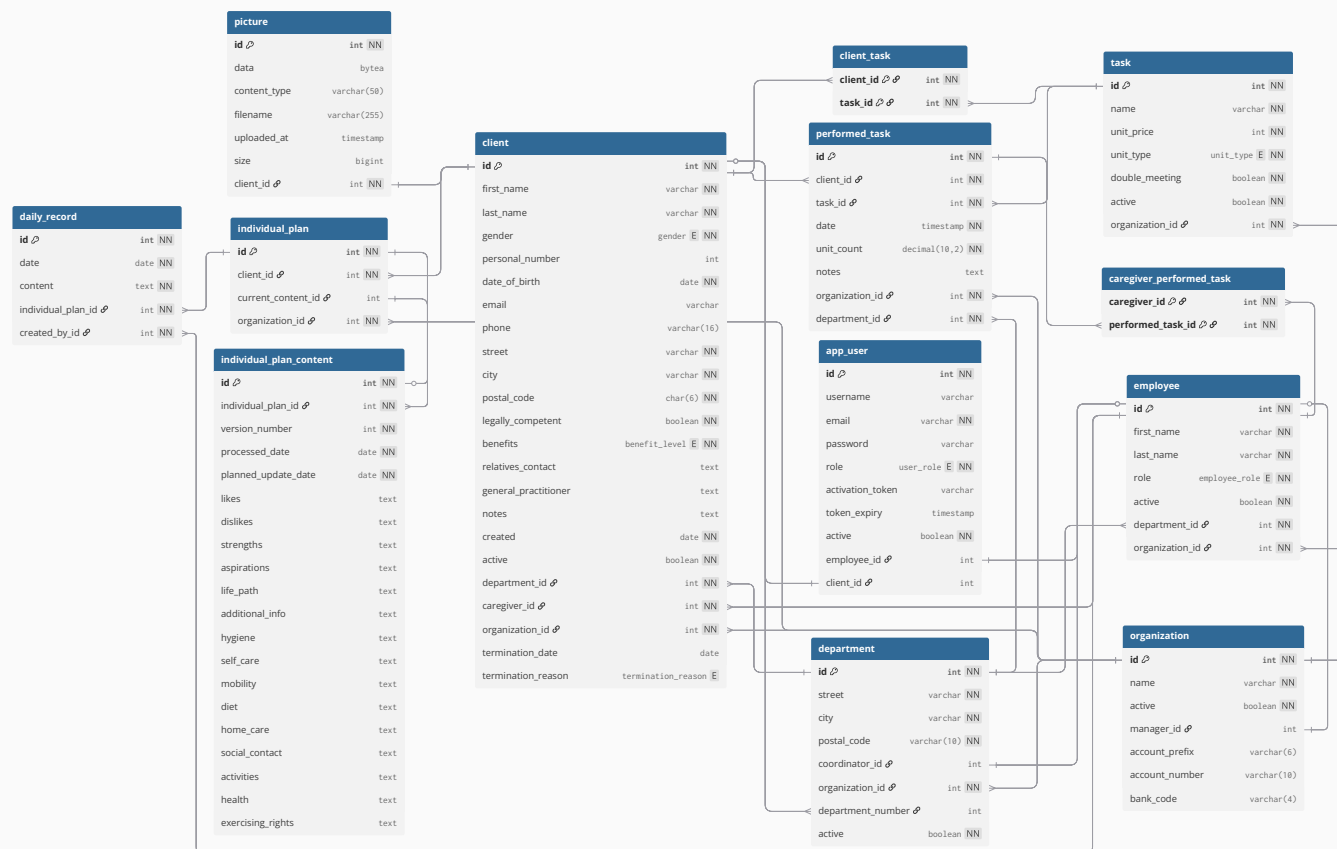
Každý zaměstnanec i klient náleží do konkrétního střediska. Klient má vždy přiřazeného klíčového pečovatele ze stejného střediska. Zaměstnanec může v systému vystupovat ve třech rolích – pečovatel, koordinátor nebo vedoucí – přičemž koordinátor střediska je uveden přímo na záznamu střediska.

Provedené úkony jsou evidovány v entitě *PerformedTask*, která odkazuje na klienta, typ úkonu (*Task*) a datum provedení. Jeden úkon může být proveden více pečovateli současně, proto je vztah mezi úkonem a pečovateli realizován jako vazební tabulka *CaregiverPerformedTask*.

Katalog úkonů (*Task*) je specifický pro každou organizaci a definuje název, cenu a způsob účtování úkonu – hodinově, kusově, v kilogramech nebo kilometrech.

Každý klient má právě jeden individuální plán péče (*IndividualPlan*). Plán je verzovaný – každá verze je uložena jako samostatný záznam *IndividualPlanContent* s číslem verze a datem zpracování. Aktuální verze je označena odkazem *current_content_id* přímo na záznamu plánu. K plánu jsou průběžně přidávány denní záznamy (*Daily-Record*), které dokumentují průběh péče.

Přihlašovací údaje jsou odděleny do entity *AppUser*, která odkazuje buď na zaměstnance, nebo na klienta. Toto oddělení umožňuje spravovat přístupové údaje nezávisle na ostatních datech a deaktivovat přístup bez ztráty historických záznamů.



Obrázek 5.1: ER diagram datového modelu systému CareTracker

5.3 Návrh uživatelského rozhraní

Uživatelské rozhraní bylo navrženo s důrazem na jednoduchost a přehlednost. Vzhledem k tomu, že pečovatelé zadávají úkony zpravidla prostřednictvím mobilního zařízení v terénu, je rozhraní plně responzivní a přizpůsobené různým velikostem obrazovky. Aplikace podporuje světlý i tmavý režim zobrazení.

5.3.1 Přihlašovací stránka

Nepřihlášený uživatel je přivítán úvodní stránkou (obrázek 5.2), která stručně představuje klíčové funkce systému – evidenci klientů, záznam provedené péče a generování přehledů. Odkaz na přihlášení je umístěn v navigační liště. Přihlašovací formulář vyžaduje uživatelské jméno a heslo.



Obrázek 5.2: Úvodní stránka aplikace CareTracker

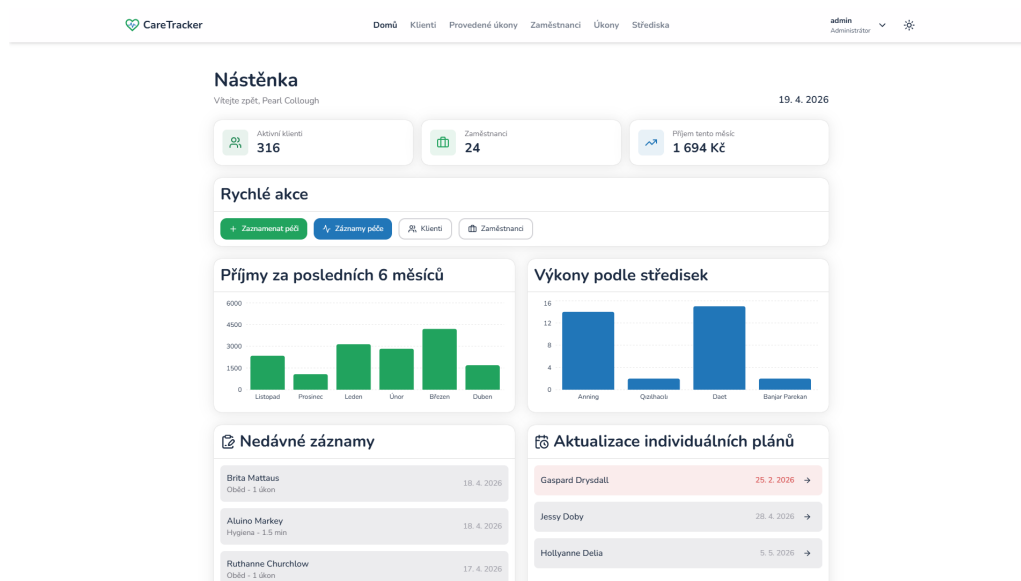
5.3.2 Nástěnka

Po přihlášení je uživatel přesměrován na nástěnku, která zobrazuje přehled nejdůležitějších informací přizpůsobený roli přihlášeného uživatele.

Pečovatel vidí tři osobní statistiky: počet úkonů, které zaznamenal v aktuálním měsíci, počet svých přiřazených klientů a celkový počet odpracovaných hodin v měsíci. Graf zobrazuje vývoj počtu provedených úkonů za posledních šest měsíců. Tlačítka rychlých akcí umožňují okamžitě zaznamenat nový úkon nebo přejít na

seznam vlastních klientů. Nástěnka pečovatele záměrně nezobrazuje finanční údaje – ty jsou v kompetenci koordinátora a administrátora.

Koordinátor vidí počet úkonů a klientů ve svém středisku a příjem za aktuální měsíc. Administrátor má navíc přehled počtu zaměstnanců a příjmů organizace jako celku (obrázek 5.3). Všem třem rolím je společná spodní část nástěnky se seznamem nedávných záznamů péče a přehledem klientů, jejichž individuální plán vyžaduje aktualizaci – prošlé položky jsou zvýrazněny červeně.



Obrázek 5.3: Nástěnka systému z pohledu administrátora

5.3.3 Evidence klientů

Stránka evidence klientů (obrázek 5.4) zobrazuje tabulkový seznam všech klientů dostupných danému uživateli podle jeho role. Tabulka umožňuje vyhledávání podle jména a filtrování podle střediska, pečovatele nebo stavu klienta. Přidání nového klienta se provede prostřednictvím modálního formuláře (obrázek 5.5), který obsahuje osobní údaje, adresu, přiřazení ke středisku a pečovateli a doplňující informace jako stupeň závislosti nebo kontakt na příbuzné.

5.3.4 Provedené úkony

Stránka provedených úkonů zobrazuje seznam všech zaznamenaných úkonů s možností filtrování podle období, klienta, pečovatele nebo střediska. Záznam nového úkonu se provede prostřednictvím modálního formuláře (obrázek 5.6), kde uživatel

CareTracker

Domů **Klienti** Provedení úkonů Zaměstnanci Úkony

caretracker

🔍 Vyhledat podle jména...

Status: ▼ Příčinnost: ▼ Tisk: ▼

Column 70 items

JMÉNO	POHLAVÍ	ADRESA	STŘEDISKO	PEČOVATEL	AČK
Agatha Adkhard	Muž	726 Fleming Place, Sacramento	Anning	Dinah Tittman	⋮
Wu Alid	Muž	28077 Crescent Oaks Terrace, Redox	Anning	Loret Lynock	⋮
Felding Antonio	Žena	25 Mallory Park, Tempe	Anning	Renado Corbier	⋮
Carie Atwood	Žena	0 High Crossing Road, Holden	Anning	Yolanda Seamen	⋮
Fla Burdbrook	Žena	37 Cony Center, Pingping	Anning	Dinah Tittman	⋮
Adrian Beardsow	Žena	68080 Hudson Court, Shangkou	Anning	Dinah Tittman	⋮
Gregory Berceli	Muž	6386 Vernon Place, Madison	Anning	Renado Corbier	⋮
Ellary Betour	Žena	62588 San Vito, Matco de Camaraghe	Anning	Dinah Tittman	⋮
Janette Bile	Žena	7433 Ronald Reagan Junction, Asento de Gacila	Anning	Loret Lynock	⋮
Jessie Bignace	Muž	4 Michigan Junction, Mikkilampoo	Anning	Loret Lynock	⋮
Jeffrey Brumstein	Žena	629 Summers Avenue, Puerto Ignaci	Anning	Loret Lynock	⋮
Kenny Calisher	Muž	294 Hansen Trail, Babel	Anning	Yolanda Seamen	⋮
Michael Cuff	Muž	7 Crowhardt Junction, Kufarik	Anning	Dinah Tittman	⋮
Frances Dancs	Žena	14 Darton Trail, Phomany-Bramura	Anning	Yolanda Seamen	⋮
Dukey Davell	Muž	30564 brother Avenue, Nahu	Anning	Renado Corbier	⋮
Hollyanne Delta	Žena	5 Blake Point, Buzibiana	Anning	Dinah Tittman	⋮
Katana Darby	Žena	7322 Memorial Junction, Songianghe	Anning	Loret Lynock	⋮
Jessy Doby	Žena	93807 Maywood Alley, Lupen	Anning	Renado Corbier	⋮
Eartha Demegen	Muž	637 Gualterri Circle, Gapsoli	Anning	Renado Corbier	⋮
Gisquard Drysdale	Muž	3508 Sachjen Junction, Hedlam	Anning	Yolanda Seamen	⋮
Brenda Dunnington	Žena	40214 Shuplin Hill, Kugang	Anning	Dinah Tittman	⋮
Benton Dunsweath	Muž	64453 Stephen Drive, Liawang	Anning	Dinah Tittman	⋮
Gusa Eozcan	Žena	015 Michigan Junction, Zhanghu	Anning	Loret Lynock	⋮

Obrázek 5.4: Seznam klientů

vybere klienta, typ úkonu, zadá rozsah a datum. Cena úkonu je vypočtena automaticky a zobrazena v náhledu ještě před uložením.

5.3.5 Individuální plán péče

Stránka individuálního plánu (obrázek 5.7) zobrazuje strukturovaný profil klienta rozdělený do tematických sekcí – osobní cíle, zvyky, zdravotní stav, mobilita, stravování a další. Plán je verzovaný; uživatel může přepínat mezi jednotlivými verzemi prostřednictvím výběru v záhlaví stránky. V dolní části stránky jsou zobrazeny denní záznamy dokumentující průběh péče s možností přidání nového záznamu.

Obrázek 5.5: Formulář pro přidání nového klienta

Obrázek 5.6: Formulář pro záznam provedeného úkonu

CareTracker

Domů

Klienti

Provedené úkony

Zaměstnanci

Úkony

Statistika

admi
Administrator

< Zpět

Individuální plán

Verze 2 (aktuální)
Verze 2 - 16. 9. 2025 (aktuální)
Verze 1 - 8. 3. 2025

Základní informace

A Obecné informace

Jméno

Jenny

Příjmení

Díky

Datum narození

23. 9. 1951

Odkazy výsledů služeb

16. 5. 2017

Středisko

Arening

Klíčový pracovník

Rivada Center

A Adresa bydliště

Ulice a číslo popisné

33907 Maywood Alley

Město

Luton

PSČ

44237

Kontakt na příbuzné

-

Doplňující informace

Připraven na péči

Léčebná

Omezení ve výživnosti

Ne

Osobní profil

Co mám ráda, co mě baví, co je pro mě důležité

Váření, četba beletrie, vířivky

Co nemám ráda, co mi vadí

Hluk, rychlá tempo, stres

Co na mě mají lidé rádi, mě slábné stránky

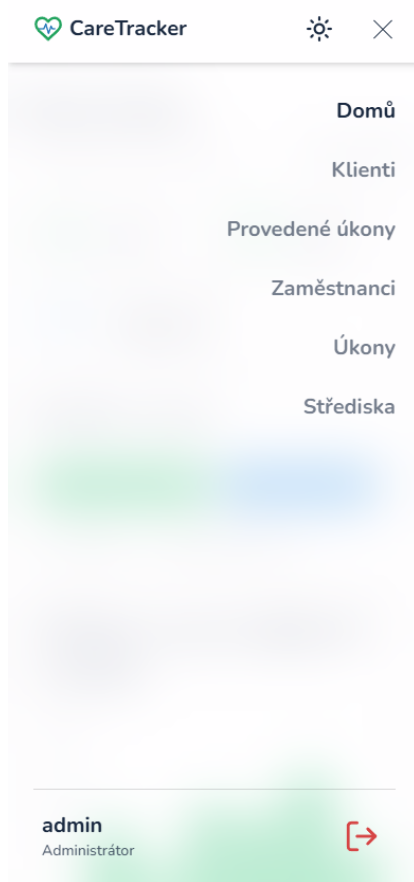
Teplo, slábné stránky

Obrázek 5.7: Individuální plán péče klienta

5.3.6 Responzivní design a mobilní rozhraní

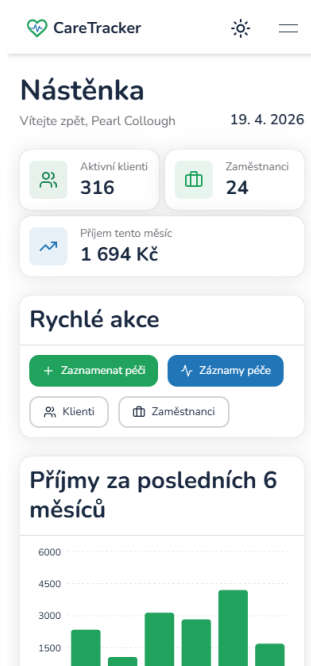
Vzhledem k tomu, že pečovatelé zadávají úkony přímo v domácnosti klienta prostřednictvím mobilního zařízení, bylo responzivní přizpůsobení rozhraní jedním z klíčových požadavků. Na menších obrazovkách se rozvržení stránky přizpůsobuje dostupné šířce – vícesloupcové layouty se skládají do jednoho sloupce a tabulky se zjednodušují tak, aby zobrazovaly jen nejdůležitější informace.

Navigační lišta se na mobilních zařízeních transformuje na rozbalovací menu přístupné přes ikonu v pravém horním rohu obrazovky (obrázek 5.8). Menu zobrazuje seznam všech dostupných sekcí a v dolní části jméno a roli přihlášeného uživatele s tlačítkem pro odhlášení.



Obrázek 5.8: Navigační menu na mobilním zařízení

Nástěnka a seznam klientů jsou na mobilní obrazovce plně funkční (obrázky 5.9 a 5.10). Formulář pro záznam úkonu (obrázek 5.11) je přizpůsoben ovládání dotykem – všechna pole jsou dostatečně velká a formulář je přístupný přímo z nástěnky jedním klepnutím.



Obrázek 5.9: Nástěnka na mobilu

The screen shows a list of clients with the following details:

- Header:** Search bar "Hledat podle jména...", filter icon, and "Přidat +" button.
- Summary:** "Celkem 70 klientů" (Total 70 clients).
- Table:**

JMÉNO	AKCE
Agosto Ackhurst	⋮
Nils Akid	⋮
Fielding Antonio	⋮
Carce Atwood	⋮
Pia Barnbrook	⋮
Adan Beardow	⋮
Gregory Beceril	⋮
Ellary Betser	⋮
Jennette Bice	⋮
Jessee Bogace	⋮
Jeffrey Brunstein	⋮
Kinny Collister	⋮

Obrázek 5.10: Seznam klientů na mobilu

The form for entering a new action includes the following fields:

- Klient:** Nils Akid
- Úkon:** Hygiena
- Počet (min):** 45
- Pečovateli:** Pearl Collough
- Datum:** 19. 04. 2026 19:05
- Cena:** 90 Kč
- Poznámky:** (Text area for notes)
- Buttons:** "Reset" and "Uložit úkon" (Save action).

Below the form, a list of recent actions is visible:

- Nealy Farris: Hygiena
- Jessy Doby: Úklid
- Nils Akid: Prádlo
- Nils Akid: Nákup
- Hollyanne Delia: Hygiena

Obrázek 5.11: Formulář úkonu na mobilu

5.4 Závěr návrhu

Na základě analýzy a specifikace byl navržen systém CareTracker jako třívrstvá webová aplikace s jasně odděleným frontendem, backendem a databází. Datový model pokrývá všechny klíčové entity pečovatelské organizace a jejich vzájemné vztahy. Uživatelské rozhraní bylo navrženo s důrazem na jednoduchost a použitelnost na mobilních zařízeních, přičemž se přizpůsobuje roli přihlášeného uživatele. Implementace systému vycházející z tohoto návrhu je popsána v následující kapitole.

Tato kapitola popisuje implementaci klíčových částí systému CareTracker pokrývající všechny funkční požadavky FR-01 až FR-08 definované v kapitole 3. Pozornost je věnována především netriviálním nebo architektonicky zajímavým aspektům – vrstvené architektuře backendu, zabezpečení přístupu, dynamickému filtrování dat, verzování individuálních plánů, generování PDF dokumentů a architektuře frontendu.

6.1 Architektura backendu

Backend je rozdělen do čtyř vrstev, které na sebe navazují a mají jasně vymezenou odpovědnost. Vrstvená architektura s REST API oddělujícím backend od frontendu přímo naplňuje požadavek NFR-04.

Controller vrstva přijímá HTTP požadavky a vrací odpovědi. Každý kontroler obsluhuje jednu doménu (klienti, úkony, zaměstnanci apod.) a je označen anotací `@RestController`. Kontroler neprovádí žádnou business logiku – pouze předává data do servisní vrstvy a vrací výsledek.

Service vrstva obsahuje veškerou business logiku. Zde probíhá ověřování oprávnění, výpočty, sestavování dotazů a koordinace mezi repozitáři. Metody určené pouze pro čtení jsou označeny anotací `@Transactional(readOnly = true)`, metody provádějící změny pak `@Transactional`.

Repository vrstva zajišťuje přístup k databázi prostřednictvím Spring Data JPA. Každý repozitář dědí z `JpaRepository` a `JpaSpecificationExecutor`, což umožňuje jak základní CRUD operace, tak pokročilé dynamické dotazy.

Mapper vrstva převádí entity na přenosové objekty (DTO) a zpět. Toto oddělení zajišťuje, že interní struktura databázových entit není přímo vystavena v API.

Pro ilustraci průchodu požadavkem slouží vytvoření nové verze individuálního plánu péče. Kontroler přijme požadavek a předá ho servisní vrstvě:

Zdrojový kód 6.1: Controller – aktualizace individuálního plánu

```
1 @PostMapping("/{clientId}/individual-plan")
```



```

2  @PreAuthorize("hasAnyRole('SUPERADMIN','ADMIN','COORDINATOR','CAREGIVER')")
3  public ResponseEntity<IndividualPlanDTO> updateIndividualPlan(
4      @PathVariable Long clientId,
5      @RequestBody IndividualPlanContentRequestDTO contentDTO) {
6      IndividualPlanDTO updated =
7          clientService.updateIndividualPlan(clientId, contentDTO);
8      return ResponseEntity.ok(updated);
9  }
10

```

Servisní metoda ověří přístup uživatele ke klientovi, najde existující plán, určí nejvyšší číslo stávající verze a vytvoří verzi novou. Odkaz `currentContent` je přesměrován na ni, přičemž předchozí verze zůstávají v databázi zachovány jako historie:

Zdrojový kód 6.2: Service – vytvoření nové verze individuálního plánu

```

1  @Transactional
2  public IndividualPlanDTO updateIndividualPlan(
3      Long clientId, IndividualPlanContentRequestDTO contentDTO) {
4      validateClientAccess(clientId);
5
6      IndividualPlan plan = individualPlanRepository.findById(
7          clientId)
8          .orElseThrow(() -> new ResourceNotFoundException(
9              "Individuální plán klienta nebyl nalezen"));
10
11      validateIndividualPlanUpdateAccess(plan.getClient());
12
13      Integer maxVersion = plan.getContentVersions().stream()
14          .map(IndividualPlanContent::getVersionNumber)
15          .max(Integer::compareTo)
16          .orElse(0);
17
18      IndividualPlanContent newVersion = new IndividualPlanContent();
19      newVersion.setIndividualPlan(plan);
20      newVersion.setVersionNumber(maxVersion + 1);
21      mapContentFields(newVersion, contentDTO);
22
23      plan.setCurrentContent(newVersion);
24      plan.getContentVersions().add(newVersion);
25
26      return individualPlanMapper.toDTO(individualPlanRepository.save(
27          plan));
28  }
29

```

6.2 Zabezpečení a řízení přístupu

Autentizace je implementována pomocí Spring Security s přihlašováním prostřednictvím formuláře. Po úspěšném přihlášení server vytvoří session uloženou v cookie s platností 7 dní (remember-me). Frontend při každém požadavku tuto cookie automaticky přiloží. Při odpovědi s kódem 401 (nepřihlášen) frontend uživatele automaticky odhlásí a přesměruje na přihlašovací stránku. Tato sekce popisuje implementaci požadavků FR-02 a NFR-01.

Oprávnění k jednotlivým operacím jsou definována přímo na metodách kontrolerů pomocí anotace `@PreAuthorize`. Například čtení katalogu úkonů je povoleno všem přihlášeným zaměstnancům, zatímco jeho správa je omezena pouze na administrátory a superadministrátory:

Zdrojový kód 6.3: Příklady řízení přístupu pomocí `@PreAuthorize`

```

1  @GetMapping
2  @PreAuthorize("hasAnyRole('SUPERADMIN', 'ADMIN', 'COORDINATOR', 'CAREGIVER')")
3  public ResponseEntity<List<TaskDTO>> getTasks(...) { ... }

5  @PostMapping
6  @PreAuthorize("hasAnyRole('SUPERADMIN', 'ADMIN')")
7  public ResponseEntity<TaskDTO> createTask(...) { ... }
8

```

6.2.1 Filtrování dat podle role

Samotná anotace `@PreAuthorize` chrání endpoint před neoprávněným voláním, ale nestačí k zajištění toho, aby uživatel viděl pouze data své organizace nebo střediska. Tuto odpovědnost nese abstraktní třída `BaseRoleFilteringService`, od níž dědí všechny servisní třídy pracující s organizačními daty.

Metoda `calculateRoleBasedFilters()` na základě role přihlášeného uživatele sestaví sadu filtrů, která je následně předána do databázového dotazu. Logika je přímočará – superadministrátor může použít libovolné parametry z požadavku, administrátor má vždy přiřazenu svou organizaci, koordinátor a pečovatel jsou omezeni výhradně na své středisko:

Zdrojový kód 6.4: Výpočet filtrů podle role přihlášeného uživatele

```

1  protected RoleBasedFilters calculateRoleBasedFilters(
2      Long requestedOrganizationId, List<Long> requestedDepartmentIds
3  ) {

4      User user = getCurrentUser();
5      UserRole role = user.getRole();

```

```
7      if (role == UserRole.SUPERADMIN) {
8          return new RoleBasedFilters(
9              requestedOrganizationId, requestedDepartmentIds);

11     } else if (role == UserRole.ADMIN) {
12         Long userOrgId = user.getEmployee().getOrganization().getId();
13         return new RoleBasedFilters(userOrgId, requestedDepartmentIds);

15     } else if (role == UserRole.COORDINATOR || role == UserRole.
16 CAREGIVER) {
17         Long userOrgId = user.getEmployee().getOrganization().getId();
18         Long userDeptId = user.getEmployee().getDepartment().getId();

19         if (requestedDepartmentIds != null
20             && !requestedDepartmentIds.isEmpty()
21             && !requestedDepartmentIds.contains(userDeptId)) {
22             return RoleBasedFilters.noAccess();
23         }

25         return new RoleBasedFilters(
26             userOrgId, Collections.singletonList(userDeptId));
27     }

29     throw new SecurityException("Nemáte oprávnění zobrazit tato data");
30 }
31
```

Výsledný objekt `RoleBasedFilters` obsahuje `organizationId` a seznam `departmentIds`, které jsou předány do JPA Specification. Pokud koordinátor požaduje data střediska, ke kterému nemá přístup, metoda vrátí speciální hodnotu `noAccess()` a servisní vrstva vrátí prázdný seznam bez přístupu k databázi.

6.2.2 Pozvánkový systém

Noví uživatelé nejsou do systému přidáváni s heslem – místo toho jim je při vytvoření účtu odeslán aktivační e-mail s odkazem obsahujícím jednorázový token. Po kliknutí na odkaz uživatel nastaví vlastní heslo a účet se aktivuje. Token má omezenou platnost a je po použití zneplatněn. Tímto způsobem jsou do systému zváni zaměstnanci i klienti.

Stejná logika jednorázového tokenu je využita i při obnově zapomenutého hesla – uživatel požádá o reset, systém mu zašle e-mail s odkazem obsahujícím nový token a po jeho použití si uživatel nastaví nové heslo.

6.3 Dynamické filtrování pomocí JPA Specifications

Přehledy úkonů, klientů a zaměstnanců (FR-03, FR-04, FR-06) podporují filtrování podle více kritérií současně – například podle střediska, pečovatele, klienta a období zároveň. Počet aktivních filtrů se může lišit případ od případu, takže sestavování dotazu pevně zakódovanými podmínkami by bylo nepraktické.

Řešením je vzor JPA Specifications, který umožňuje skládat podmínky dotazu dynamicky za běhu aplikace. Každá podmínka je samostatný objekt implementující rozhraní `Specification<T>`. Podmínky lze řetěžit metodou `.and()`, přičemž pokud je daný filtr prázdný, podmínka se jednoduše vynechá. Výsledná specifikace je předána metodě `findAll()` repozitáře:

Zdrojový kód 6.5: Sestavení dynamického dotazu pomocí Specifications

```
1 Specification<PerformedTask> spec = PerformedTaskSpecifications
2   .withFilters(organizationId, departmentIds, caregiverIds,
3               clientId, month, year);

5 List<PerformedTask> results = performedTaskRepository.findAll(spec);
6
```

Abstraktní základní třída `BaseSpecifications` poskytuje sdílené pomocné metody pro filtrování podle organizace, střediska, stavu a dalších společných atributů, aby se předešlo duplicitnímu kódu napříč různými doménami.

6.4 Verzování individuálních plánů péče

Individuální plán péče je pro každého klienta jeden, avšak jeho obsah se v čase mění. Systém uchovává celou historii změn prostřednictvím verzování – každá úprava plánu vytvoří novou verzi, přičemž předchozí verze zůstávají zachovány (FR-07).

Entita `IndividualPlan` obsahuje seznam všech verzí (`IndividualPlanContent`) a odkaz na aktuální verzi (`currentContent`). Každá verze má přiřazené pořadové číslo. Při vytvoření první verze je pořadové číslo nastaveno na 1. Při každé následující úpravě je nalezeno nejvyšší existující číslo verze, nová verze dostane číslo o jedna vyšší a odkaz `currentContent` je přesměrován na ni. Starší verze zůstávají v databázi nedotčeny.

Denní záznamy (`DailyRecord`) jsou vedeny odděleně od verzování – jsou vázány přímo na plán, nikoli na konkrétní verzi. Záznamy tak přetrvávají napříč všemi verzemi plánu a tvoří kontinuální deník péče o klienta.

Na frontendu uživatel přepíná mezi verzemi prostřednictvím výběru v záhlaví stránky. Výběr aktuální verze stáhne ze serveru pouze metadata jednotlivých verzí

(datum zpracování, číslo verze) a plný obsah je načten až po výběru konkrétní verze – tím se minimalizuje objem přenášených dat.

6.5 Generování PDF

Fakturační podklady (FR-08) jsou generovány na straně serveru jako PDF dokumenty. Proces sestává ze tří kroků.

V prvním kroku jsou z databáze načteny provedené úkony za zadané období a klienta, seskupeny podle typu úkonu a jsou vypočteny celkové ceny.

Ve druhém kroku je vygenerován QR kód pro platbu. Kód je sestavován dotazem na externí API službu Paylibo [28], které na základě bankovního spojení organizace, celkové částky a variabilního symbolu vrátí QR kód jako obrázek. Variabilní symbol je odvozen z roku, měsíce a osobního čísla klienta, což umožňuje jednoznačně párovat platby.

Ve třetím kroku je HTML šablona naplněna daty pomocí Thymeleaf a převedena do formátu PDF knihovnou Flying Saucer. Šablona je navržena jako tisknutelný dokument – používá tabulkové rozvržení a vlastní písmo Nunito, které je přibaleno přímo v aplikaci, aby byl výsledný dokument vizuálně konzistentní bez závislosti na systémových písmech serveru. Výsledný PDF dokument je odeslán do prohlížeče, kde si ho uživatel může stáhnout nebo rovnou vytisknout.

Pečovatelská služba		Duben 2026
Nils Akid	Nar.: 7.10.1944	Adresa: 28077 Crescent Oaks Terrace, 23862 , Reutov
Provedené úkony		
Název úkonu	Počet	Cena
Nákup	45.0 min	88 Kč
Prádlo	1.42 kg	153 Kč
		Celkem: 241 Kč
		
..... Datum Převzal Podpis		

Obrázek 6.1: Ukázka vygenerovaného fakturačního podkladu

6.6 Architektura frontendu

Frontend je postaven na principu oddělení zodpovědností mezi stránky a custom hooks, přičemž responzivní design a mobilní použitelnost naplňují požadavek NFR-02.

Stránky (složka `pages/`) jsou React komponenty zodpovědné výhradně za vykreslení uživatelského rozhraní – tabulky, modální okna, formuláře. Neobsahují žádnou logiku pro práci s API.

Custom hooks (složka `hooks/`) zapouzdřují veškerou logiku pro komunikaci se serverem – načítání dat, odesílání požadavků, správu stavu načítání a zobrazování chybových hlášení. Každá doménová oblast má vlastní hook (např. `useClients`, `usePerformedTasks`, `useIndividualPlan`). Stránka hook pouze zavolá a pracuje s jeho výstupem.

Díky tomuto rozdělení je kód stránek přehledný a logiku načítání dat lze snadno znovu použít na více místech. Aby bylo možné zrušit probíhající požadavek při rychlém přepínání filtrů nebo stránek, využívají hooks `AbortController` – při každém novém požadavku je předchozí automaticky zrušen.

Filtrování a stav stránek jsou persistovány v URL parametrech pomocí `useSearchParams`. Uživatel tak může sdílet odkaz na konkrétní filtrovaný přehled nebo se vrátit na stejné zobrazení po obnovení stránky.

Uživatelské rozhraní je dále rozděleno do znovupoužitelných komponent uložených ve složce `components/`. Modální okna, formuláře a sdílené prvky jsou implementovány jako samostatné komponenty, které stránky pouze skládají dohromady. Díky tomu je každá část rozhraní definována na jednom místě a může být použita na více stránkách bez duplikace kódu.

6.7 Kontejnerizace

Celé prostředí aplikace je definováno v souboru `docker-compose.yml` a sestává ze tří služeb: backend, frontend a databáze (PostgreSQL), čímž je naplněn požadavek NFR-03. Citlivé údaje jako přihlašovací údaje k databázi jsou odděleny do souboru `.env`, který není součástí verzovacího systému.

Backend využívá Spring Boot DevTools pro automatické načtení změn bez nutnosti restartu celého kontejneru – po rekompilaci zdrojových souborů příkazem `make compile` se backend restartuje automaticky. Frontend využívá Vite vývojový server s okamžitou aktualizací změn v prohlížeči. Zdrojové soubory obou služeb jsou sdíleny s kontejnerem prostřednictvím Docker volumes, díky čemuž se změny v kódu projeví okamžitě bez nutnosti znovu sestavovat obraz.

Data databáze jsou ukládána do pojmenovaného volume `pgdata`, který přetrvává i po zastavení kontejnerů. Celé prostředí lze spustit jediným příkazem `make up`

a zastavit příkazem `make down`.

Testování systému CareTracker probíhalo dvěma způsoby – automatizovanými testy klíčových částí kódu a průběžným manuálním testováním v průběhu celého vývoje.

7.1 Automatizované testy

7.1.1 Backendové testy

Backendové testy jsou implementovány pomocí frameworku JUnit 5 [29] a knihovny Mockito [30], která umožňuje nahradit závislosti (databáze, repozitáře) simulovanými objekty. Testy jsou rozděleny do tří úrovní.

Testy entity ověřují výpočetní logiku třídy `PerformedTask`. Metoda `calculatePrice()` musí správně vypočítat cenu úkonu pro všechny čtyři typy účtování – hodinový, kusový, kilogramový a kilometrový – včetně správného zaokrouhlení u hodinového typu, kde je výsledek přepočítáván z minut.

Testy servisní vrstvy ověřují business logiku nezávisle na databázi. Testovány jsou zejména scénáře řízení přístupu – například že klient vidí pouze vlastní úkony, koordinátor nedostane data cizího střediska nebo že pečovatel nemůže smazat úkon, ke kterému není přiřazen.

Testy kontrolerů ověřují HTTP vrstvu pomocí nástroje `MockMvc`, který simuluje HTTP požadavky bez spouštění skutečného serveru. Testovány jsou správné návratové kódy – nepřihlášený uživatel obdrží odpověď 401, pečovatel pokoušející se vytvořit klienta obdrží 403 a požadavek na neexistující záznam vrátí 404.

7.1.2 Frontendové testy

Frontendové testy jsou implementovány pomocí frameworku Vitest [31]. Testovány jsou pomocné utility funkce ve složce `utils/` – formátovací funkce (`formatPhoneNumber`, `formatPostalCode`, `formatDate` aj.) a funkce pro řazení dat (`sortByKey`, `sortStrings`). Testy ověřují správné formátování vstupů včetně hraničních případů jako `null` hodnoty, krátké řetězce nebo čeština s diakritikou.

7.1.3 Výsledky automatizovaných testů

Tabulka 7.1: Výsledky automatizovaných testů

Prostředí	Sada testů	Počet testů	Úspěšné	Neúspěšné
Backend	PerformedTaskTest	6	6	0
Backend	PerformedTaskServiceTest	5	5	0
Backend	ClientServiceTest	3	3	0
Backend	ClientControllerTest	5	5	0
Frontend	sorting.test.js	14	14	0
Frontend	formatters.test.js	23	23	0
Celkem		56	56	0

7.2 Manuální testování

Hlavní metodou ověření funkčnosti systému bylo průběžné manuální testování v průběhu celého vývoje. Každá nově implementovaná funkce byla okamžitě odzkoušena v prohlížeči, přičemž byly testovány jak standardní scénáře, tak hraniční případy a chybové stavy.

7.2.1 Testovací prostředí

Testování probíhalo na lokálním prostředí spuštěném pomocí Docker Compose. Aplikace byla testována v prohlížeči Brave (jádro Chromium). Responzivní chování rozhraní bylo ověřeno jak pomocí vývojářských nástrojů prohlížeče simulujících mobilní obrazovku, tak přímo na fyzickém zařízení iPhone 14 Pro, kde bylo otestováno ovládání dotykem, responzivní rozvržení a plynulost přechodu mezi stránkami.

7.2.2 Testované scénáře

Testování pokrývalo hlavní uživatelské scénáře pro každou roli systému.

Pro roli *administrátora* bylo ověřeno zakládání středisek a zaměstnanců, správa katalogu úkonů, odesílání pozvánek a přehled dat celé organizace na nástěnce.

Pro roli *koordinátora* bylo ověřeno zakládání a správa klientů, vytváření a verzování individuálních plánů péče, přidávání denních záznamů, záznam provedených úkonů a generování fakturačních podkladů ve formátu PDF.

Pro roli *pečovatele* bylo ověřeno zobrazení přidělených klientů, rychlý záznam provedeného úkonu prostřednictvím mobilního zařízení a přístup k individuálním plánům péče.

Pro roli *klienta* bylo ověřeno zobrazení vlastních záznamů a QR kódu pro platbu.

Zvláštní pozornost byla věnována testování izolace dat mezi organizacemi a středisky – bylo ověřeno, že uživatel jedné organizace nemá přístup k datům jiné organizace a koordinátor nevidí data cizího střediska.

7.3 Závěr testování

Automatizované testy pokrývají kritické části systému – výpočetní logiku, řízení přístupu a HTTP vrstvu. Manuální testování ověřilo funkčnost všech hlavních uživatelských scénářů pro každou roli. Během testování byly odhaleny a opraveny drobné chyby v chování uživatelského rozhraní a v logice filtrování dat.

Možnosti dalšího rozvoje

8

Před případným komerčním nasazením by bylo nutné aplikaci doplnit o výkaznictví pro Ministerstvo práce a sociálních věcí. Poskytovatelé sociálních služeb jsou ze zákona povinni vykazovat statistická data do registru poskytovatelů, a bez této integrace by systém nemohl plně nahradit stávající komerční řešení.

Mezi přínosnými rozšířeními by bylo plánování směn, které bylo v průzkumu označeno jako jedna z nejnáročnějších administrativních oblastí. Systém v současné době eviduje provedené úkony zpětně, avšak koordinátor nemá nástroj pro předběžné plánování kdy a kdo navštíví kterého klienta.

Dalším přirozeným rozšířením by byl systém push notifikací – pečovatel by mohl být upozorněn na naplánovaný úkon a koordinátor by dostával upozornění na individuální plány vyžadující aktualizaci.

Z dlouhodobého hlediska by vývoj nativní mobilní aplikace zlepšil uživatelský komfort pro pečovatele v terénu – zejména přirozenější ovládání dotykem a lepší integraci se systémem zařízení, například oznámeními. Webová aplikace již v současné podobě podporuje instalaci jako progresivní webová aplikace (PWA¹), díky níž si ji uživatel může přidat na plochu mobilního zařízení a používat bez nutnosti otevírat prohlížeč.

¹Progressive Web App – webová aplikace instalovatelná na plochu zařízení, která se chová podobně jako nativní aplikace.

Cílem této bakalářské práce bylo navrhnout a implementovat webovou aplikaci CareTracker pro správu pečovatelských služeb. Aplikace pokrývá klíčové procesy pečovatelské organizace – evidenci klientů a zaměstnanců, zaznamenávání provedených úkonů, správu individuálních plánů péče včetně verzování a generování fakturačních podkladů ve formátu PDF. Systém je navržen jako víceuživatelská platforma s hierarchií přístupových rolí podporující správu více organizací a jejich středisek v rámci jedné instance.

Před zahájením vývoje byla provedena analýza problémové domény, existujících řešení a dotazníkový průzkum mezi poskytovateli pečovatelských služeb. Průzkum potvrdil, že správa individuálních plánů péče a ruční zpracování výkazů představují největší administrativní zátěž, a tyto oblasti se staly prioritou při návrhu systému.

Výsledkem práce je funkční webová aplikace připravená ke spuštění prostřednictvím jediného příkazu díky kontejnerizaci nástrojem Docker. Uživatelské rozhraní je responzivní a plně použitelné na mobilních zařízeních, což bylo ověřeno jak vývojářskými nástroji prohlížeče, tak na fyzickém zařízení. Všechny funkční požadavky definované ve specifikaci byly implementovány.

Dotazník – Průzkum správy pečovatelských služeb

A

Následující dotazník byl rozeslán poskytovatelům pečovatelských služeb v dubnu 2026 za účelem získání podkladů pro analýzu problémové domény. Průzkumu se zúčastnilo 11 organizací; výsledky jsou uvedeny vždy pod příslušnou otázkou, odpovědi respondentů jsou označeny R1–R11 v pořadí, v jakém byly doručeny.

1. Kolik klientů má Vaše organizace?

- do 20
- 21 – 50
- 51 – 100
- více než 100

Výsledky:

	21–50	51–100	více než 100
Počet organizací	1	4	6

2. Kolik zaměstnanců má Vaše organizace?

- do 5
- 6 – 15
- 16 – 30
- více než 30

Výsledky:

	do 5	6–15	16–30	více než 30
Počet organizací	1	5	2	3

3. Kolik středisek má Vaše organizace?

- 1
- 2 – 3
- více než 3

Výsledky:

	1	2–3	více než 3
Počet organizací	6	3	2

4. Jakým způsobem aktuálně evidujete úkony? (*možno označit více*)

- Specializovaný software
- Tabulkový procesor (Excel, Google Sheets apod.)
- Papírová evidence
- Jiné

Výsledky:

Způsob evidence	Počet organizací
Pouze specializovaný software	5
Specializovaný software + papírová evidence	5
Pouze papírová evidence	1

5. Pokud používáte software, uveďte název (např. Cygnus):
(*otevřená odpověď*)

Výsledky:

Respondent	Uvedený software
R1	James edition (systém pana Zajíce)
R2	Pečovatelka Deluxe
R3	Petr Zajíc – James
R4	Zajíc
R5	Cygnus
R6	Cygnus
R7	–
R8	Chytrá organizace (přechod na Cygnus)
R9	Pečovatelská služba (pan Zajíc)
R10	eQuip
R11	eQuip

6. Jaké jsou Vaše přibližné měsíční náklady na software?

- 0 Kč
- do 2 000 Kč
- 2 000 – 10 000 Kč
- více než 10 000 Kč
- Nechci uvádět

Výsledky:

Náklady	Počet organizací
0 Kč	1
do 2 000 Kč	4
2 000–10 000 Kč	5
Nechci uvádět	1

7. Co je pro vás administrativně nejnáročnější? (možno označit více)

- Individuální plány péče
- Podklady pro fakturaci
- Plánování směn
- Sdílení dat v terénu
- Jiné

Výsledky (možno označit více):

Oblast	Počet organizací
Plánování směn	6
Podklady pro fakturaci	5
Individuální plány péče	4
Sdílení dat v terénu	2

8. Jak jste spokojeni s Vaším současným řešením?
(stupnice 1 – 5, kde 1 = velmi nespokojen, 5 = velmi spokojen)

Výsledky: R1: 4, R2: 3, R3: 3, R4: 3, R5: 3, R6: 4, R7: 1, R8: 1, R9: 4, R10: 4, R11: 4

(průměr: 3,1)

9. Jaké problémy vám současný způsob evidence způsobuje?
(otevřená odpověď)

Výsledky:

R1: Žádné.

R2: Dvojí evidence je časově náročnější.

R3: Občasné technické problémy.

R4: Nemáme problémy.

R5: Propojení vedoucích a terénních pracovníků.

R6: Chybějící data z přechozího softwaru; ručně se zpracovávají vyúčtování pro klienty v externím programu.

R7: Časová náročnost.

R8: Software neustále hlásí špatné připojení a výpadky, nevyhovující plánování péče.

R9: Nedostatečné plánování směny v programu.

R10: Občasný výpadek systému.

R11: Software postrádá některé funkce (např. podpora AI).

10. Kolik hodin měsíčně zabere ruční zpracování výkazů?
(otevřená odpověď)

Výsledky: R1: 16, R2: 8, R3: 0*, R4: 40, R5: 3**, R6: –, R7: 12, R8: 10, R9: 10, R10: 0, R11: 6

* Respondent uvedl, že výkazy ručně nezpracovávají.

** Respondent uvedl „3 hodiny jeden pracovník“.

11. Vedete individuální plány péče digitálně nebo papírově?

- Digitálně – je součástí dříve zmíněného softwaru
- Digitálně – používáme separátní software
- Papírově

Výsledky:

Způsob vedení	Počet organizací
Digitálně – součást softwaru pro úkony	4
Digitálně – separátní software	2
Papírově	3
Přechodné období (papír i digitálně)	1
Neuvedeno	1

12. Mají pečovatelé při výkonu práce k dispozici chytrý telefon s přístupem k internetu?
- Ano
 - Ne

Výsledky: Ano – 7 organizací, Ne – 4 organizace.

13. Potřebujete propojení s některým jiným softwarem? *(možno označit více)*
- Účetní systém (Pohoda, Money apod.)
 - Výkaznictví pro MPSV / registr poskytovatelů
 - Jiný typ softwaru
 - Nepotřebuji integraci

Výsledky (možno označit více):

Typ integrace	Počet organizací
Výkaznictví pro MPSV / registr poskytovatelů	8
Účetní systém	4
Jiný typ softwaru	1
Nepotřebuji integraci	1

Respondentům byl před následující sekci předložen stručný popis vyvíjené aplikace CareTracker – její funkcionalita, podpora více středisek a rolí, responzivní webové rozhraní bez nutnosti instalace.

14. Uvítali byste nasazení takového systému ve vaší organizaci?

- Ano
- Ne
- Nevím

Výsledky: Ano – 6, Ne – 2, Nevím – 3.

15. Co považujete za největší přínos oproti vašemu současnému řešení?
(otevřená odpověď)

Výsledky:

- R1: Žádný (spokojeni se stávajícím systémem).
- R2: Jednoduchost pro uživatele a automatické generování podkladů k vyúčtování.
- R3: Nevím.
- R4: Nelze posoudit bez vyzkoušení.
- R5: Nevím (organizace již používá Cygnus).
- R6: Nemohu posoudit.
- R7: Rychlost a přehlednost.
- R8: Systém lépe přizpůsobený potřebám pečovatelské služby.
- R9: Využitelné plánování směn.
- R10: Nevím.
- R11: Integrace AI, generování obsahu dle požadavků kraje.

16. Jaká měsíční cena by byla pro Vaši organizaci akceptovatelná?

- do 500 Kč
- 500 – 1 500 Kč
- 1 500 – 3 000 Kč
- více než 3 000 Kč
- Nechci uvádět

Výsledky:

Cenové rozmezí	Počet organizací
do 500 Kč	1
500–1 500 Kč	2
1 500–3 000 Kč	6
více než 3 000 Kč	1
Nechci uvádět	1

17. Co by vás od nasazení nového systému odradilo? (*otevřená odpověď*)

Výsledky:

R1: Spokojeni se stávajícím systémem.

R2: Nevím.

R3: Migrace dat.

R4: Nic, pokud by byl systém lepší než současný.

R5: Obavy o zabezpečení.

R6: Opětovná migrace a kontrola dat.

R7: Nic.

R8: Migrace dat (organizace aktuálně přechází na jiný software).

R9: Nevím.

R10: Vysoká cena.

R11: Složitá migrace, nevole zaměstnanců.

18. Jak důležitá je pro vás možnost zadávat úkony z mobilního zařízení přímo u klienta?

(*stupnice 1 – 5, kde 1 = nedůležité, 5 = velmi důležité*)

Výsledky: R1: 3, R2: 5, R3: 5, R4: 3, R5: 5, R6: 1, R7: 4, R8: 5, R9: 5, R10: 5, R11: 4
(**průměr: 4,1**)

19. Preferujete z hlediska bezpečnosti / GDPR:

- Cloudové řešení (data na serveru poskytovatele)
- On-premise (data na vlastním serveru organizace)
- Nemám preference / nevím

Výsledky:

Preference	Počet organizací
Cloudové řešení	6
On-premise (vlastní server)	2
Nemám preference / nevím	3

Uživatelská příručka –

Spuštění aplikace

B

Tato příloha popisuje postup instalace a spuštění aplikace CareTracker na lokálním počítači.

B.1 Systémové požadavky

Pro spuštění aplikace jsou potřeba tyto nástroje:

- **Docker** (verze 24 nebo novější) včetně Docker Compose – ke stažení na docker.com,
- **GNU Make** – součást většiny linuxových distribucí; na Windows je dostupný např. přes WSL.

B.2 Konfigurace

Před prvním spuštěním je nutné vytvořit soubor `.env` v kořenovém adresáři projektu. Soubor definuje přihlašovací údaje k databázi – hodnoty lze zvolit libovolně:

Zdrojový kód B.1: Obsah souboru `.env`

```
1 POSTGRES_USER=caretracker
2 POSTGRES_PASSWORD=heslo
3 POSTGRES_DB=caretracker
4
```

Dále je nutné vytvořit soubor `frontend/.env` s adresou backendu:

Zdrojový kód B.2: Obsah souboru `frontend/.env`

```
1 VITE_API_URL=http://localhost:8080/api
2
```

Aplikace rovněž odesílá automatická e-mailová upozornění (např. o prošlých individuálních plánech). K tomu slouží soubor `backend/src/main/resources/secrets.properties`, který obsahuje přihlašovací údaje k e-mailové schránce Gmail s App Password:

Zdrojový kód B.3: Obsah souboru `secrets.properties`

```
1 spring.mail.username=vase.adresa@gmail.com
2 spring.mail.password=xxxx xxxx xxxx xxxx
3
```

Pokud odesílání e-mailů není potřeba, stačí ponechat v souboru libovolné hodnoty – aplikace nashartuje i bez platných e-mailových přihlašovacích údajů, pouze odesílání zpráv nebude funkční.

B.3 Spuštění

Po vytvoření obou souborů `.env` stačí v kořenovém adresáři projektu spustit:

Zdrojový kód B.4: Spuštění aplikace

```
1 make up
2
```

Příkaz sestaví Docker image pro backend i frontend, spustí databázový server a automaticky provede migrace schématu. Při prvním spuštění může sestavení trvat několik minut v závislosti na rychlosti připojení k internetu.

B.4 Přístup k aplikaci

Po úspěšném startu je aplikace dostupná v prohlížeči na adrese `localhost:3000`. Databáze je předvyplněna ukázkovými daty; pro každou roli existuje ukázkový účet pojmenovaný podle role (`superadmin`, `admin`, `coordinator`, `caregiver`, `client`), heslo je u všech účtů `heslo`.

B.5 Zastavení a opětovné spuštění

Aplikaci lze zastavit příkazem:

Zdrojový kód B.5: Zastavení aplikace

```
1 make down
2
```

Příkaz zastaví a odstraní kontejnery, data v databázi jsou však zachována. Opětovné spuštění se provede stejným příkazem jako při prvním startu:

Zdrojový kód B.6: Opětovné spuštění aplikace

```
1 make up
2
```

Bibliografie

1. ČESKÝ STATISTICKÝ ÚŘAD. *Ukazatele věkového složení* [tabulka]. ČSÚ, 2024. [cit. 2026-04-03]. Dostupné z: <https://vdb.czso.cz/vdbvo2/faces/cs/index.jsf?page=vystup-objekt&z=T&f=TABULKA&skupId=6512%20&katalog=33156&pvo=DEMD13aG&pvo=DEMD13aG>.
2. *Zákon č. 108/2006 Sb., o sociálních službách* [Sbírka zákonů České republiky]. 2006. [cit. 2026-04-03]. Dostupné z: https://ppropo.mpsv.cz/zakon%5C_108%5C.2006. ISSN 1211-1244.
3. IRESOFT S.R.O. *Cygnus: Komplexní informační systém pro poskytovatele pobytových, ambulantních a terénních služeb* [online]. 2024. [cit. 2026-04-03]. Dostupné z: <https://www.iscygnus.cz>.
4. PETR ZAJÍC SOFTWARE. *Pečovatelská služba* [online]. 2004. [cit. 2026-04-16]. Dostupné z: <https://www.xzajic.cz/pecovatelska-sluzba/>.
5. JOPLAN. *Joplan: Nejlepší pomocník pro individuální plánování* [online]. 2026. [cit. 2026-04-03]. Dostupné z: <https://www.joplan.cz>.
6. SOFTWARE PRODUCTION. *eQuip: Každá sociální služba potřebuje elektronickou evidenci* [online]. 2026. [cit. 2026-04-16]. Dostupné z: <https://info.equip.cz/>.
7. BALAT, Petr. *Pečovatelka Deluxe: Informační systém pro sociální služby* [online]. 2026. [cit. 2026-04-16]. Dostupné z: <https://www.softdeluxe.cz/>.
8. ORACLE CORPORATION. *Java* [online]. 2026. [cit. 2026-04-19]. Dostupné z: <https://www.java.com>.
9. VMWARE LLC. *Spring Boot* [online]. 2026. [cit. 2026-04-19]. Dostupné z: <https://spring.io/projects/spring-boot>.
10. PALLETS PROJECTS. *Flask: A Lightweight WSGI Web Application Framework* [online]. 2026. [cit. 2026-04-19]. Dostupné z: <https://flask.palletsprojects.com>.
11. VMWARE LLC. *Spring Security* [online]. 2026. [cit. 2026-04-19]. Dostupné z: <https://spring.io/projects/spring-security>.

12. VMWARE LLC. *Spring Data JPA* [online]. 2026. [cit. 2026-04-19]. Dostupné z: <https://spring.io/projects/spring-data-jpa>.
13. RED HAT, INC. *Hibernate ORM* [online]. 2026. [cit. 2026-04-19]. Dostupné z: <https://hibernate.org/orm>.
14. REDGATE SOFTWARE. *Flyway: Database Migrations Made Easy* [online]. 2026. [cit. 2026-04-19]. Dostupné z: <https://www.red-gate.com/products/flyway>.
15. THE POSTGRES SQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL: The World's Most Advanced Open Source Relational Database* [online]. 2026. [cit. 2026-04-19]. Dostupné z: <https://www.postgresql.org>.
16. META PLATFORMS, INC. *React: The Library for Web and Native User Interfaces* [online]. 2025. [cit. 2026-04-19]. Dostupné z: <https://react.dev>.
17. MDN WEB DOCS. *JavaScript* [online]. 2024. [cit. 2026-04-19]. Dostupné z: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>.
18. MICROSOFT CORPORATION. *TypeScript: JavaScript With Syntax For Types* [online]. 2026. [cit. 2026-04-19]. Dostupné z: <https://www.typescriptlang.org>.
19. EVAN YOU, VOIDZERO, INC. *Vite: The Build Tool for the Web* [online]. 2026. [cit. 2026-04-19]. Dostupné z: <https://vite.dev>.
20. TAILWIND LABS, INC. *Tailwind CSS: A Utility-First CSS Framework* [online]. 2026. [cit. 2026-04-19]. Dostupné z: <https://tailwindcss.com>.
21. NEXTUI, INC. *HeroUI* [online]. 2026. [cit. 2026-04-19]. Dostupné z: <https://www.heroui.com>.
22. REMIX SOFTWARE, INC. *React Router* [online]. 2025. [cit. 2026-04-19]. Dostupné z: <https://reactrouter.com>.
23. DOCKER, INC. *Docker: Accelerated Container Application Development* [online]. 2026. [cit. 2026-04-19]. Dostupné z: <https://www.docker.com>.
24. DOCKER, INC. *Docker Compose* [online]. 2026. [cit. 2026-04-19]. Dostupné z: <https://docs.docker.com/compose>.
25. FLYING SAUCER PROJECT. *Flying Saucer* [online]. 2026. [cit. 2026-04-19]. Dostupné z: <https://github.com/flyingsaucerproject/flyingsaucer>.
26. VMWARE, INC. *Spring Framework* [online]. 2026. [cit. 2026-04-19]. Dostupné z: <https://docs.spring.io/spring-framework/reference/integration/email.html>.
27. THE THYMELEAF TEAM. *Thymeleaf: A Modern Server-Side Java Template Engine* [online]. 2026. [cit. 2026-04-19]. Dostupné z: <https://www.thymeleaf.org>.

28. PAYLIBO S.R.O. *Paylibo – QR platby pro Českou republiku* [online]. 2026. [cit. 2026-04-20]. Dostupné z: <https://qr-platba.cz/pro-vyvojare/restful-api>.
29. JUNIT TEAM. *JUnit 5* [online]. 2026. [cit. 2026-04-24]. Dostupné z: <https://docs.junit.org/>.
30. MOCKITO CONTRIBUTORS. *Mockito: Tasty mocking framework for unit tests in Java* [online]. 2026. [cit. 2026-04-24]. Dostupné z: <https://site.mockito.org/>.
31. VITEST CONTRIBUTORS. *Vitest: A Vite-native test framework* [online]. 2026. [cit. 2026-04-24]. Dostupné z: <https://vitest.dev/>.

Seznam obrázků

3.1	Diagram případů užití	17
5.1	ER diagram datového modelu systému CareTracker	24
5.2	Úvodní stránka aplikace CareTracker	25
5.3	Nástěnka systému z pohledu administrátora	26
5.4	Seznam klientů	27
5.5	Formulář pro přidání nového klienta	28
5.6	Formulář pro záznam provedeného úkonu	28
5.7	Individuální plán péče klienta	29
5.8	Navigační menu na mobilním zařízení	30
5.9	Nástěnka na mobilu	31
5.10	Seznam klientů na mobilu	31
5.11	Formulář úkonu na mobilu	31
6.1	Ukázka vygenerovaného fakturačního podkladu	37

Seznam tabulek

1.1	Vybrané ukazatele věkového složení obyvatelstva České republiky [1] .	4
2.1	Způsob aktuální evidence úkonů	10
2.2	Administrativně nejnáročnější oblasti	11
2.3	Způsob vedení individuálních plánů péče	11
7.1	Výsledky automatizovaných testů	41

Seznam výpisů

6.1	Controller – aktualizace individuálního plánu	32
6.2	Service – vytvoření nové verze individuálního plánu	33
6.3	Příklady řízení přístupu pomocí @PreAuthorize	34
6.4	Výpočet filtrů podle role přihlášeného uživatele	34
6.5	Sestavení dynamického dotazu pomocí Specifications	36
B.1	Obsah souboru .env	52
B.2	Obsah souboru frontend/.env	52
B.3	Obsah souboru secrets.properties	53
B.4	Spuštění aplikace	53
B.5	Zastavení aplikace	53
B.6	Opětovné spuštění aplikace	53

1101001 1100001
10101100001110010 1100001
10101 10101 10101 10101



11010011101101001
01100001 10101
1110001010111 10101